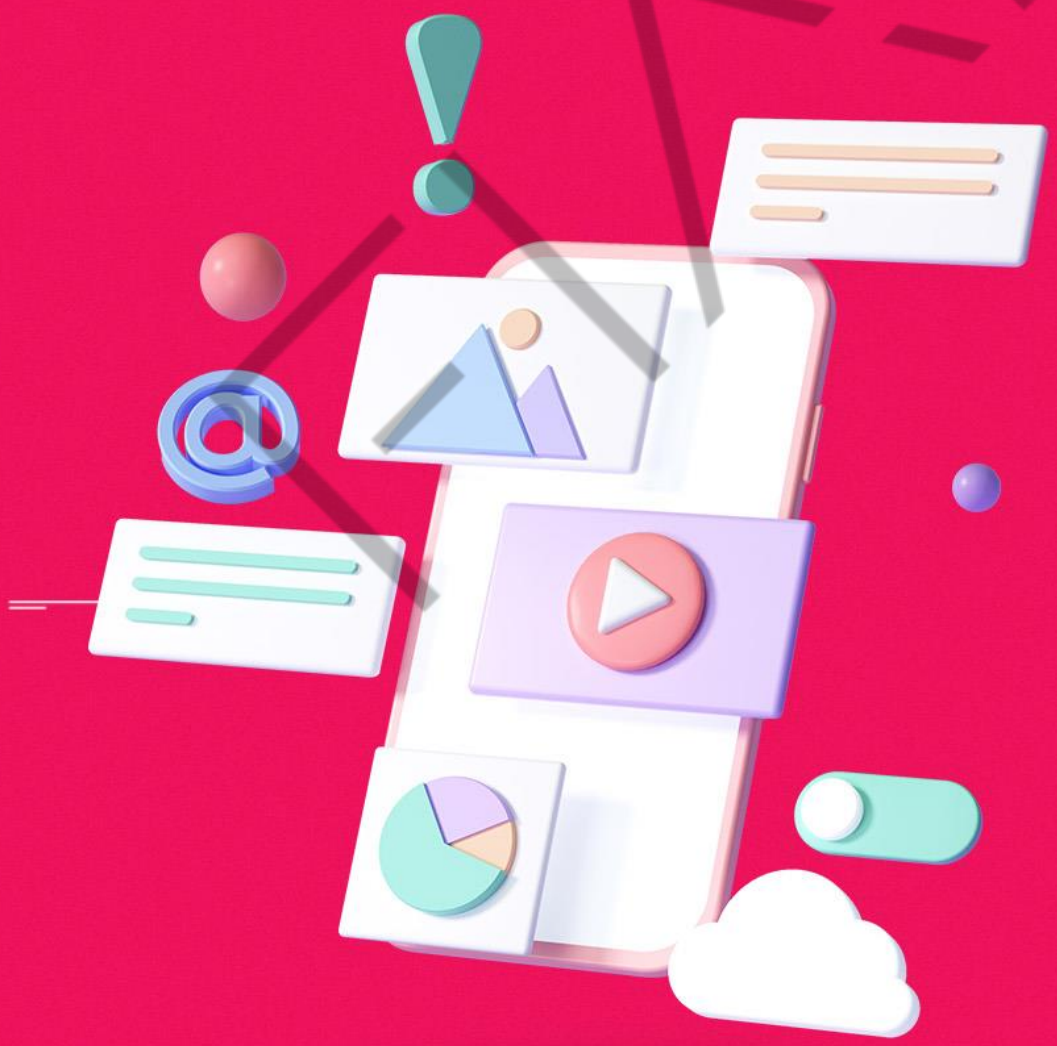
FIAP ON

IOS

XCODE ESSENTIALS



3

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Site do Xcode 7
Figura 2 – Welcome to Xcode 8
Figura 3 – Escolhendo template App 9
Figura 4 – Definindo as configurações do aplicativo 11
Figura 5 – Conhecendo a Tool bar 12
Figura 6 – Áreas do Xcode 14
Figura 7 – Library – códigos 17
Figura 8 – Library – UI 17
Figura 9 – Arquivo AppDelegate.swift 18
Figura 10 – Arquivo AppDelegate.swift 19
Figura 11 – Métodos do AppDelegate.swift e SceneDelegate.swift 20
Figura 12 – Selecionado Target iOS 12.0 21
Figura 13 – SceneDelegate versão iOS 12.0 22
Figura 14 – AppDelegate versão iOS 12.0 23
Figura 15 – Criando arquivos. Swift 24
Figura 16 – Rodando aplicativo no simulador e visualizando resultado 27
Figura 17 – Exemplo de um *storyboard* de um filme 28
Figura 18 – Exemplo de *storyboard* 29
Figura 19 – Interface do *Storyboard* 30
Figura 20 – Site de developers da Apple 31
Figura 21 – Criando conta de desenvolvedor 32
Figura 22 – Logado na área de desenvolvedor 32
Figura 23 – Entrando em Accounts e Clicando no + ... 33
Figura 24 – Configurando conta no Xcode 34
Figura 25 – Conta configurada 34
Figura 26 – Vinculando conta ao projeto 35
Figura 27 – Selecionando device 36
Figura 28 – Modo de Desenvolvimento - Desativado 37
Figura 29 – Configurando a tela inicial do app 38
Figura 30 – Inserindo Label 39
Figura 31 – Modificando as propriedades do Label 40
Figura 32 – Adicionando imagem no Assets.xcassets 41
Figura 33 – Adicionando imagem na cena 42
Figura 34 – Configurando Image View 43
Figura 35 – Aplicativo Hello Universe 44

TEXVP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Classe MyClass 25
Código-fonte 2 – Acessando a classe MyClass 26

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 A CASA DO IOS.....5
1.1 App Store e Xcode 5
1.1.1 Criando o projeto de um aplicativo 7
1.1.2 Conhecendo a interface do Xcode 11
1.1.2.1 Navigator Area 14
1.1.2.2 Editor Area 15
1.1.2.3 Utilities Area 16
1.1.2.4 Library 16
1.2 Códigos e storyboard 18
1.2.1 Arquivos .swift 18
1.2.1.1 Controle de acesso.....24
1.2.2 Storyboard.....27
1.3 Criando conta developer 30
1.4 Rodando Apps no *device* 33
1.5 Criação do App Hello Universe 37
REFERÊNCIAS.....45

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 A CASA DO IOS

Agora que nos divertimos no Playground, chegou a hora de conhecer a interface de desenvolvimento dos nossos aplicativos. Você aprendeu a “falar” em *swift* e agora vamos praticar junto aos elementos de interface gráfica. Neste capítulo, você vai aprender a navegar pelo Xcode para criar seus projetos, além de poder testá-los tanto no simulador embutido na ferramenta quanto em seu próprio *device*. Está pronto para criar o seu primeiro app em iOS? Vamos lá!

1.1 App Store e Xcode

Com o lançamento do iPhone em 2007 e seu estrondoso sucesso, a Apple percebeu a necessidade de permitir que desenvolvedores pudessem criar seus próprios aplicativos para seu recém-lançado smartphone. Um ano após seu lançamento, mais precisamente em 10 de julho de 2008, a maçã lançou um serviço centralizado que servia como uma loja virtual de aplicativos para o iPhone, a iPhone App Store ou simplesmente App Store. A partir deste momento, qualquer pessoa com o conhecimento e ferramentas corretas poderia submeter seus aplicativos para a loja e, após aprovado pelo time de revisão dos aplicativos da Apple, estaria disponível para qualquer pessoa no mundo baixá-lo em seu device.

Nesta época existia uma IDE, que é um acrônimo para *Integrated Development Environment* (Ambiente integrado de desenvolvimento), que permitia criar softwares para o Mac OSX, sistema operacional presente nos computadores da Apple. Para contextualizar melhor, uma IDE nada mais é que um software que reúne um conjunto de ferramentas para facilitar o desenvolvimento, ou seja, além de fornecer o ambiente para a edição do código-fonte do seu App, também inclui ferramentas para o gerenciamento do App, configuração das telas e toda a parte visual, controle das imagens e outros arquivos, controle da versão, automação de tarefas de depuração (debug), compilação do código-fonte, enfim, tudo que um desenvolvedor precisa para criar seu App.

Esta IDE de que estamos falando é o Xcode que no mesmo ano de lançamento do iPhone, ganhou um upgrade que permitia que fosse também utilizado para

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

desenvolvimento dos aplicativos para iOS. Desde então, o Xcode vem evoluindo ano a ano, tornando-se cada vez mais completo e robusto.

Com relação às vendas de aplicativos na App Store, antes de 2021, para qualquer faturamento a Apple ficava com 30% e os desenvolvedores com 70%. Porém, desde 2021 com o programa “Small Business Program”, a Apple reduziu a comissão para 15% como forma de ajudar desenvolvedores menores, isso para quem tem faturamento na plataforma inferior a US$ 1milhão. Para empresas médias e grandes, ainda é cobrada uma comissão de 30% em cada venda.

Para se inscrever no Small Business Program você precisa ser titular de conta no programa de desenvolvedor da Apple e inscrever-se em: https://developer.apple.com/app-store/small-business-program/.

Com o Xcode podemos desenvolver aplicativos para qualquer sistema operacional da Apple, ou seja, através da mesma IDE conseguimos criar softwares para o iOS (iPhone e iPod), iPadOS (iPad), macOS (MacBook, iMac, Mac Mini), tvOS (Apple TV) e watchOS (Apple Watch). É possível conhecer mais a respeito do Xcode acessando seu próprio site, o <https://developer.apple.com/xcode/>, que contém uma série de informações a respeito desta IDE e de como trabalhar, bem como das novidades da última versão corrente.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 1 – Site do Xcode
Fonte: Apple (2023)

1.1.1 Criando o projeto de um aplicativo

Até o presente momento da nossa trilha iOS, nós temos utilizado o Xcode apenas para criar arquivos .playground, os quais utilizamos para aprender a trabalhar com a linguagem *Swift*. Agora chegou o momento de utilizarmos nossa IDE para a criação de aplicativos iOS, porém, para isso é preciso antes conhecermos como está configurada a interface do Xcode e saber onde cada pedaço do nosso código se encaixa dentro deste ambiente de desenvolvimento.

Ao iniciarmos o Xcode nos deparamos com a sua tela de *Welcome* (tela de boas-vindas), que é o local onde podemos escolher se desejamos criar um novo projeto, clonar um projeto existente ou abrir um projeto ou arquivo local.

A opção **Create a new Xcode project** nos permite iniciar um novo projeto de App, seja para iPhone, iPad, Mac, Apple Watch ou Apple TV. A opção **Clone an existing project** permite trabalhar a partir de um código pronto e que esteja presente

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

em algum repositório. A opção **Open a project or file** permite abrir um projeto ou arquivo local que está no nosso computador.

Abra o Xcode e escolha a opção **Create a new Xcode project**.

Figura 2 – Welcome to Xcode
Fonte: Elaborado pelo autor (2023)

Na tela a seguir, há a possibilidade de escolher para qual plataforma deseja criar o seu aplicativo. Por padrão, a opção que vem marcada é iOS, que é o sistema operacional presente nos iPhones, iPods e iPads e é o que usaremos no curso, porém, você pode escolher os demais através das opções dessa aba superior.

Abaixo, na seção **Application**, o Xcode fornece uma série de templates para iniciarmos nosso projeto. Cada template foi criado para um tipo de projeto específico, como, por exemplo, o **Game**, que entrega um projeto contendo algumas classes preparadas para trabalhar com o framework SpriteKit ou SceneKit, que são os frameworks que a Apple fornece para a criação dos jogos. O template que utilizaremos no curso é o **App**, que é o mais simples, contendo apenas uma tela e sem maiores configurações. É importante saber que a partir deste template podemos criar qualquer tipo de aplicativo, ou seja, não está preso a um App simples de uma única tela, apenas te entregará um projeto contendo o mínimo necessário para que você crie qualquer aplicativo que desejar. Justamente por isso que o utilizaremos no curso. Selecione **App** e clique no botão **Next** para prosseguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

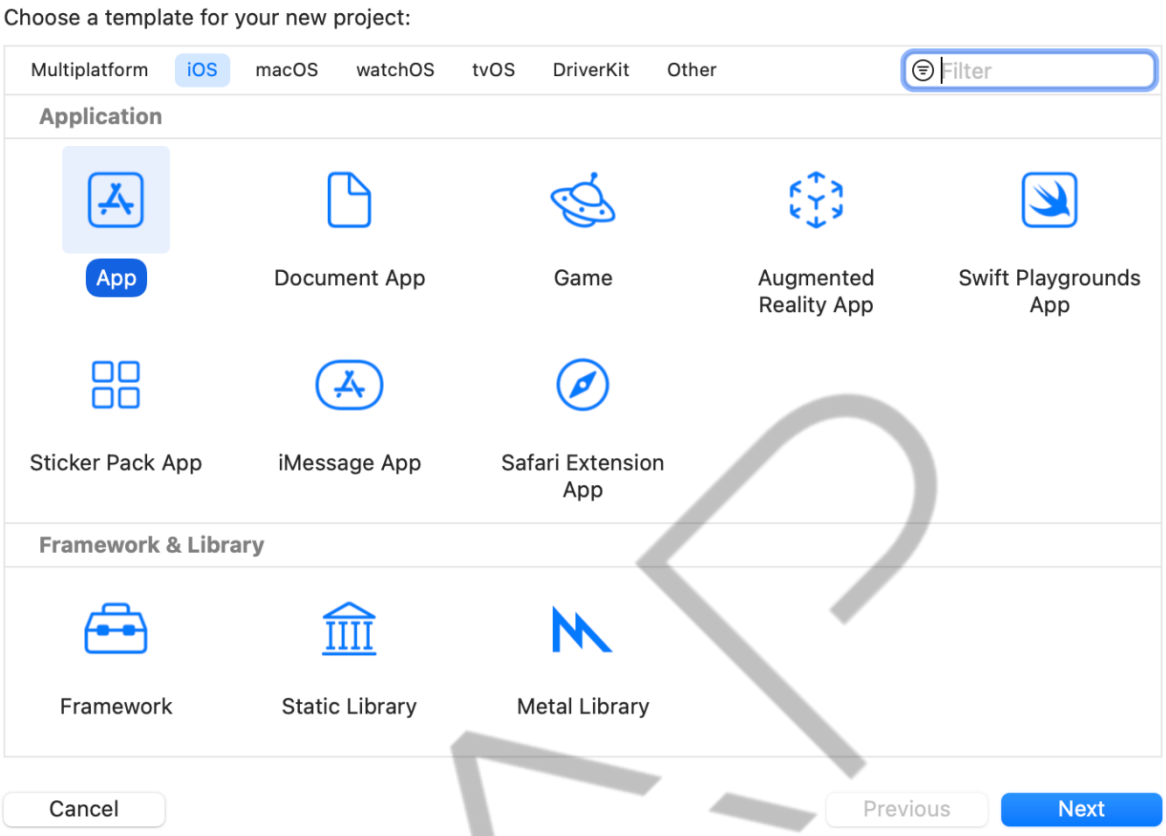


Figura 3 – Escolhendo template App
Fonte: Elaborado pelo autor (2022)

A tela seguinte é onde configuramos as principais características do nosso projeto. Estas informações são vitais para definir a "identidade" do aplicativo na App Store.

Em **Product Name** definimos o nome do nosso projeto, bem como o nome do nosso aplicativo. É aqui que configuramos que nome aparecerá para o usuário quando o App estiver instalado no seu device, além de ser também o nome do projeto em si. Vamos chamar nosso primeiro App de **MeuApp**.

Em **Team** nós definimos qual a conta de desenvolvedor será utilizada no desenvolvimento do aplicativo. Vamos deixar esta opção como **None** por enquanto, contudo iremos configurá-la em outro momento desta trilha.

O campo **Organization Identifier** é um dos mais importantes e define parte do que será o **Bundle Identifier** do seu App. Bundle Identifier (a opção que aparece logo abaixo) é o identificador único do seu aplicativo na loja da Apple, e serve para diferenciar seu App dos demais presentes na loja. Pense no Bundle Identifier como o RG do seu aplicativo para a Apple, assim, deve ser único. É formado pela junção do campo Organization Identifier e do Product Name, por isso que costuma se definir o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Organization Identifier como sendo o domínio reverso da empresa por trás do aplicativo. Dessa forma, mesmo que exista na loja outro aplicativo com o mesmo nome que o seu, terá um Bundle Identifier diferente, pois o Organization Identifier será diferente. Preencha este campo com **br.com.fiap**, que é o domínio reverso (domínio invertido) da FIAP (fiap.com.br). Perceba que nosso Bundle Identifier ficou **br.com.fiap.MeuApp**.

Em **Interface** definimos qual tecnologia utilizaremos para desenvolver nossas telas. Por padrão, a opção SwiftUI está selecionada. SwiftUI é o novo *framework* de UI desenvolvido pela Apple, que conta com o padrão declarativo para desenvolver telas e componentes. Para esta trilha, utilizaremos a opção **Storyboard**, que representa o *framework* UIKit e que ainda é predominantemente utilizado no mercado de trabalho.

Por último, certifique-se de que a linguagem selecionada seja *Swift,* que é a linguagem que utilizaremos nos nossos aplicativos. Temos também a opção de marcarmos Objective-C, a primeira linguagem usada no desenvolvimento de apps iOS, porém não trabalharemos com essa linguagem em nossa trilha.

Deixem as últimas opções desmarcadas (Use Core Data, Host in CloudKit e Include Tests) e clique em **Next**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

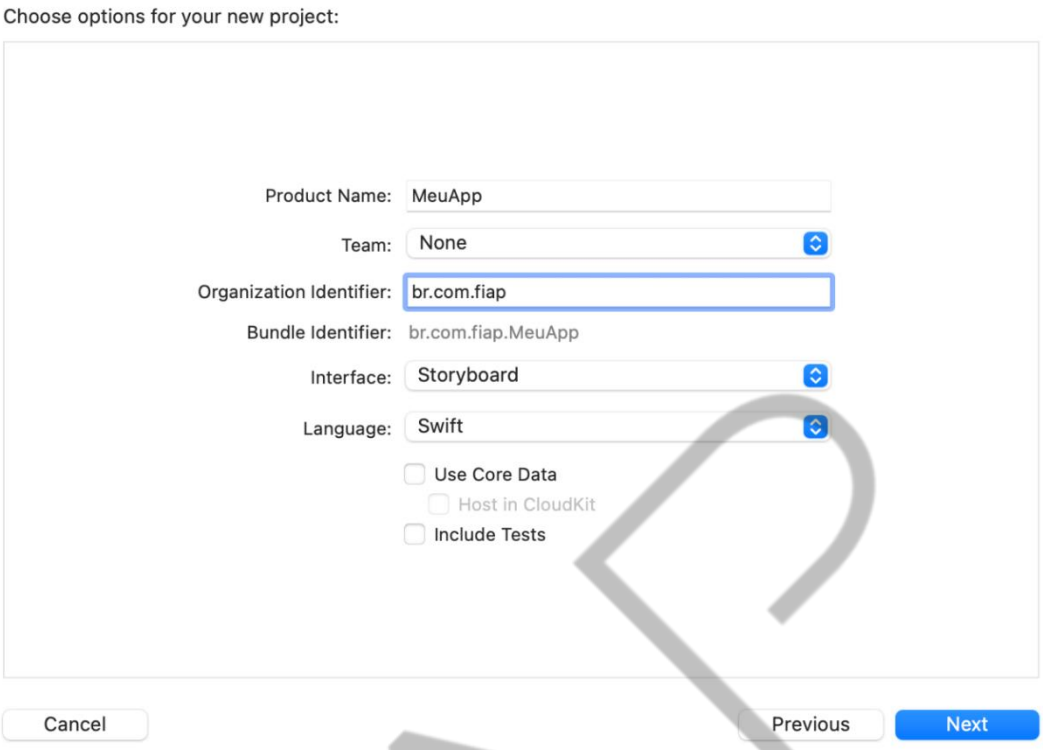


Figura 4 – Definindo as configurações do aplicativo
Fonte: Elaborado pelo autor (2022)

Por último, escolha onde deseja salvar o seu projeto e clique em **Create**. Após isto, terá seu aplicativo configurado e pronto para ser desenvolvido!

1.1.2 Conhecendo a interface do Xcode

Você está nesse momento visualizando o seu projeto devidamente configurado e pronto para ser desenvolvido. Sempre que criamos um novo aplicativo, o Xcode é inicializado na tela de configurações do seu aplicativo, já que é o conteúdo central que você está vendo. Entretanto, vamos analisar e entender como a interface do Xcode é dividida e conhecer seus principais painéis.

Na área superior encontra-se a **Tool bar**, ou a barra de ferramentas, uma área que contém uma série de botões que nos ajudam a organizar melhor os demais painéis, bem como executar nosso App ou até mesmo verificar o que o Xcode está executando naquele momento. Do lado esquerdo temos os botões de **Run** e **Stop**, que servem para que possamos rodar nosso App seja no simulador ou mesmo em nosso device e interromper sua execução.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A escolha de onde o aplicativo irá rodar será feita pelo painel localizado ao lado do botão de Stop, chamado de **Run Destination**. Ao clicar nesta opção, você pode selecionar onde quer que o seu App seja executado, dentre uma lista de simuladores que vêm instalados junto com o Xcode ou até mesmo o seu device, caso esteja plugado no seu computador e caso seu projeto tenha uma conta de desenvolvedor configurada (veremos como fazer isto em outro momento). É importante ressaltar que não precisa possuir um iPhone ou iPad para desenvolver seus aplicativos. A maioria das funcionalidades de um App podem ser testadas no simulador, o que facilita muito o desenvolvimento para pessoas que não possuem os devices da Apple. Aproveite e selecione o iPhone 14 Pro ou 14 Pro Max como simulador desejado.

Na parte central da Tool bar existe um painel chamado de **View Activity**, que nos informa que operação o Xcode está executando naquele momento, além de nos mostrar os possíveis erros ou avisos presentes no nosso projeto. É sempre bom ficar de olho nesse painel, pois nos mantém informados do que está acontecendo com nosso projeto enquanto desenvolvemos.

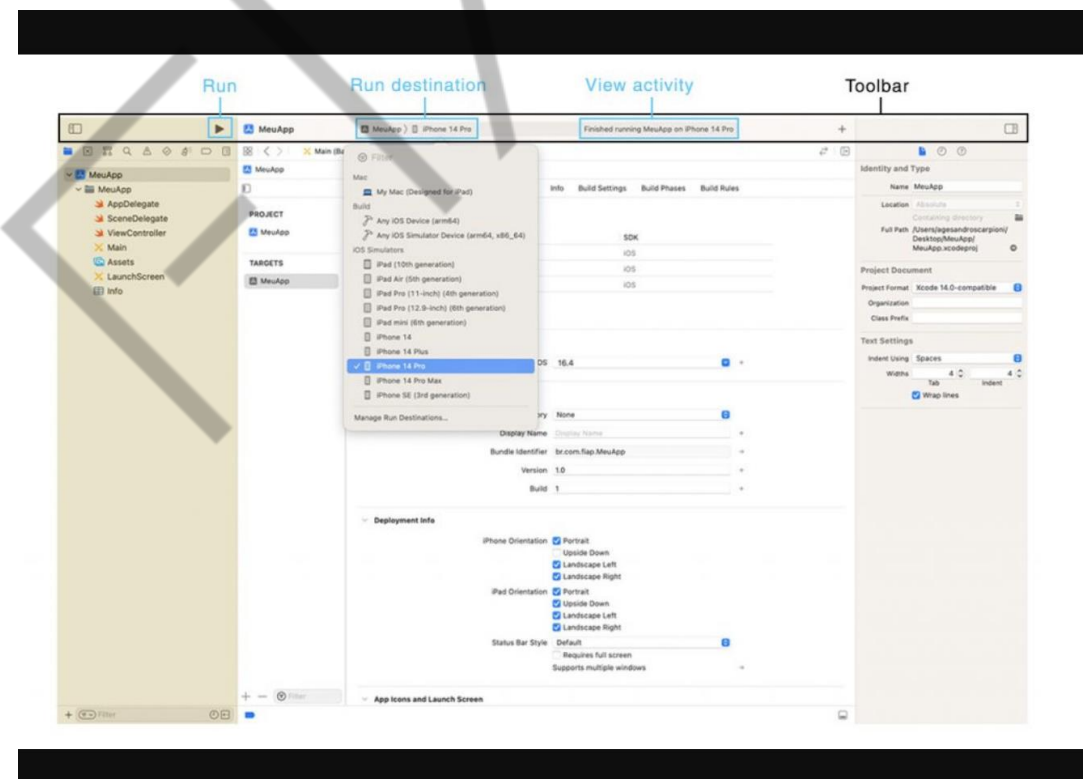


Figura 5 – Conhecendo a Tool bar
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Próximo ao canto superior direito, temos dois botões que formam a **Configure Editor Area.** Através deles podemos configurar como a nossa Editor Area será dividida. Falando em **Editor Area**, essa é a área central, onde passamos a maior parte do tempo quando estamos desenvolvendo nossos aplicativos. É nela que trabalhamos os códigos-fonte do nosso projeto, bem como implementamos nossas telas, verificamos os ajustes do projeto, enfim, como o próprio nome diz: é a área de edição do Xcode. Curioso saber que a Editor Area é sensível ao contexto, o que significa que o seu conteúdo depende do que estiver selecionado no momento. Se escolhermos algum arquivo de código (arquivos do tipo *.swift,* por exemplo ViewController) aparecerá o código contido neste arquivo. Se selecionarmos o nome do projeto, mostrará as configurações do mesmo, se escolhermos um arquivo do tipo .storyboard como por exemplo o Main, será exibido o conteúdo da tela selecionada.

Ao clicarmos no botão esquerdo da Configure Editor Area abriremos o Adjust Editor Option, um novo menu em forma de popup, onde podemos escolher entre várias opções de configuração para nossa Edit Area. Uma das opções desse menu é o **Assistant**, muito útil quando estamos trabalhando com tela e precisamos vincular elementos visuais variáveis criados em nosso código. Ao clicar no botão da direita do Configure Editor Area podemos adicionar uma nova Edit Area com conteúdo independente mostrando seu próprio Assistant por exemplo ou comandos do Editor Option como citamos anteriormente.

No canto superior direito, temos um botão que esconde/mostra o painel mais à direita, chamado de **Utilities Area**, onde podemos editar propriedades dos arquivos ou elementos selecionados. No canto superior esquerdo, temos outro botão que esconde/mostra a **Navigator Area**, que é a área localizada do lado esquerdo, onde temos acesso, dentre outras coisas, aos arquivos do nosso projeto. Experimente clicar nos botões para visualizar os painéis aparecendo e desaparecendo da interface. Lembre-se de utilizá-los caso você precise de mais espaço em tela para trabalhar seu código ou as telas do seu aplicativo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

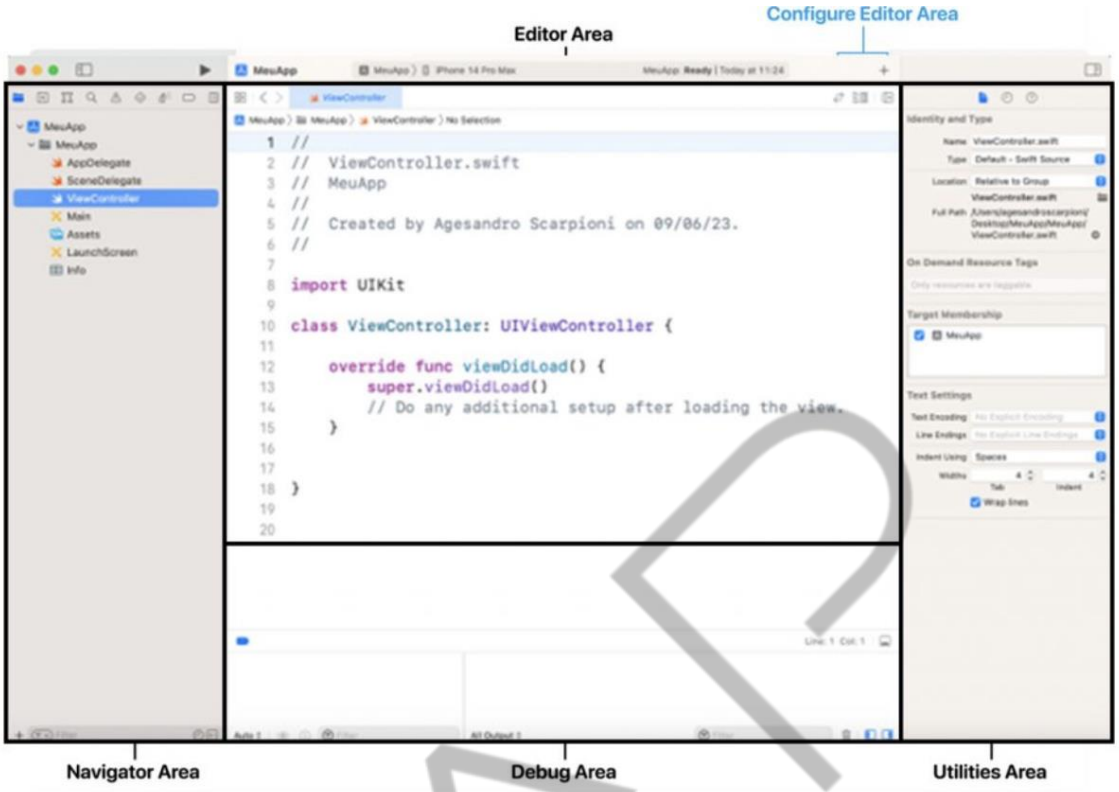


Figura 6 – Áreas do Xcode
Fonte: Elaborado pelo autor (2023)

1.1.2.1 Navigator Area

Vamos falar sobre estas áreas que estamos mostrando e escondendo, começando pela **Navigator Area**. A Navigator Area é a área que se encontra mais à esquerda e possui várias subáreas que chamamos de **Navigators**. São nove ao todo, porém vamos focar no principal que é o **Project Navigator,** que nos dá acesso aos arquivos presentes no nosso projeto. É uma representação da pasta onde criamos nosso projeto, e através dele podemos selecionar arquivos, excluir, adicionar, criar arquivos etc. Em nosso projeto atual podemos ver que temos uma pasta central chamada **MeuApp** e uma outra pasta (uma espécie de pasta virtual) chamada também **MeuApp**. A pasta que nos interessa, que é a **MeuApp,** vem com sete arquivos, sendo três arquivos de código-fonte (**AppDelegate.swift, SceneDelegate.swift** e **ViewController.swift**), a extensão .swift atualmente fica oculta. Por curiosidade, para ver a extensão você teria que clicar sobre o AppDelegate e dar em “enter”, só assim é possível visualizar a extensão, isso vale também para o Main, Assets, LaunchScreen e Info, dois arquivos que representam a parte visual (telas) do app (**Main.storyboard** e **LaunchScreen.storyboard**), um outro arquivo que na verdade é também uma outra

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

pasta, a **Assets.xcassets** que nos dá acesso a uma área onde podemos adicionar imagens e outros recursos visuais ao projeto e por fim o **Info.plist** que é o arquivo que contém as principais configurações do nosso App (todas as configurações que fizemos no início, durante a criação do projeto, estão também registradas neste arquivo).

Todos estes arquivos foram criados automaticamente pelo Xcode quando selecionamos o template App, e cada um tem o seu papel dentro do projeto. Experimente selecionar cada um deles para ver como a Editor Area se modifica para mostrar o conteúdo do arquivo.

Aproveitando, dê uma olhada nos demais *navigators*. Na sequência, temos o **Source Control Navigator** (para controle de versão - git), o **Symbol Navigator** (nos fornece uma lista das classes, métodos e variáveis do projeto), **Find Navigator** (permite buscar por determinado trecho de código ou arquivo), **Issue Navigator** (lista de erros ou alertas), **Test Navigator** (listagem dos testes unitários e de UI), **Debug Navigator** (mostra os processos que estão ocorrendo quando debugamos um app), **Breakpoint Navigator** (listagem de todos os breakpoints) e por último o **Report Navigator** (relatório de testes).

1.1.2.2 Editor Area

A Editor Area é a principal área do Xcode, onde passamos a maior parte do tempo trabalhando. É uma área *content-sensitive*, ou em português, sensível ao conteúdo ou contexto, que significa que se adequa ao conteúdo que foi selecionado. Em outras palavras, quando selecionamos um arquivo *.swift,* o código-fonte será apresentado, ao selecionar um arquivo *.jpg,* a imagem correspondente será exibida. Note que o nome da Content Area é alterado de acordo com o que está sendo apresentado. Quando estamos trabalhando com código-fonte, esta área passa a se chamar de **Source Editor**. Quando selecionamos o projeto (aquele nome em azul no tipo da Navigator Area), mostrará as configurações do seu projeto e seu nome muda para **Project Editor**. Com relação às telas do aplicativo neste caso os arquivos Main e LaunchScreen, que poderão ser configuradas por meio de arquivo.storyboard, ao selecioná-los, a Editor Area passa a se chamar **Interface Builder**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.2.3 Utilities Area

Do lado direito da interface do Xcode temos a **Utilities Area**, que como o próprio nome dá a entender, é uma área de utilitários. Assim como a Editor Area, também é uma área sensível ao contexto, apresentando informações diferentes de acordo com o tipo de arquivo ou componente selecionado.

No topo da tela, temos uma barra chamada **Inspector Bar** que, dependendo do arquivo selecionado, pode conter até sete botões (quando selecionamos o arquivo Main.storyboard). Se estamos com algum arquivo (que não é .storyboard) selecionado, esta área nos mostra basicamente as informações do seu nome, localização, tipo etc. Entretanto, a Utilities Area brilha mesmo quando estamos editando as telas do nosso App, ou seja, quando trabalhamos com arquivo .storyboard. Selecione o arquivo Main.storyboard que teremos sete botões para trabalhar, nos dando a possibilidade de editar as propriedades da tela ou de qualquer componente adicionado na mesma.

Agora que conhecemos sobre a interface do Xcode e seus diversos painéis e áreas, vamos olhar com mais calma para os arquivos que trabalharemos ao longo dos nossos projetos que são os arquivos contendo código-fonte (arquivos .swift) e as telas do aplicativo (arquivos .storyboard).

1.1.2.4 Library

Uma novidade no Xcode 10 foi a inclusão do botão Library no Configure Editor Area, ele é o símbolo “+” que aparece nesse setor. Com ele podemos acessar blocos de códigos pré-carregados, e incluí-los aos nossos códigos, bem como todos os elementos de interface disponíveis para habitar uma tela. O conteúdo da Library também é content-sensitive, dessa maneira, quando estamos em um arquivo de códigos (.swift), o conteúdo exibido será o de blocos de códigos, ao passo que, quando selecionamos um arquivo de interface (.storyboard), seu conteúdo muda para elementos de UI.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

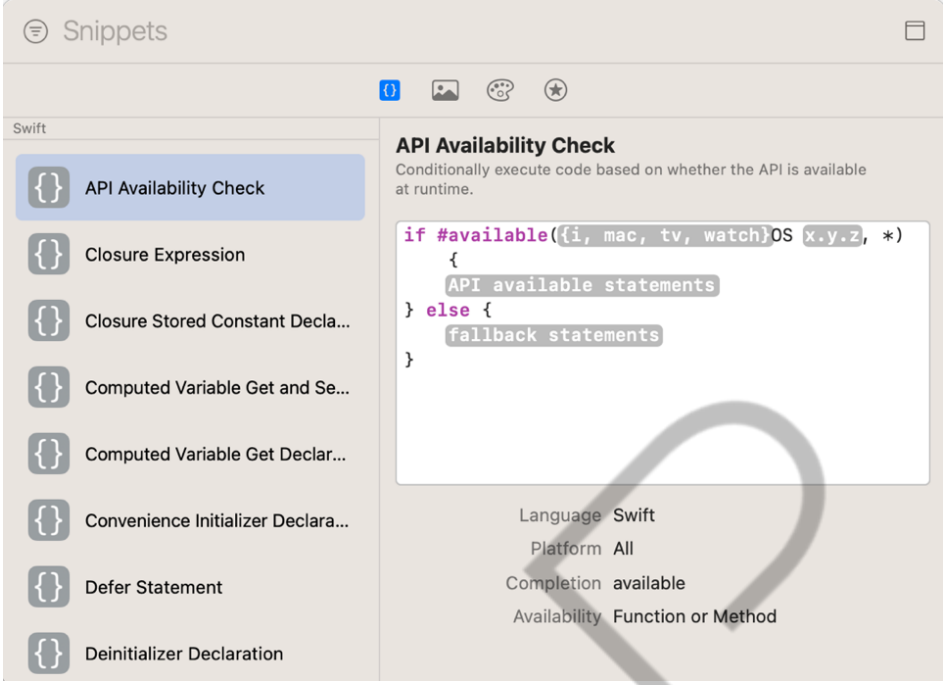


Figura 7 – Library – códigos
Fonte: Elaborado pelo autor (2023)

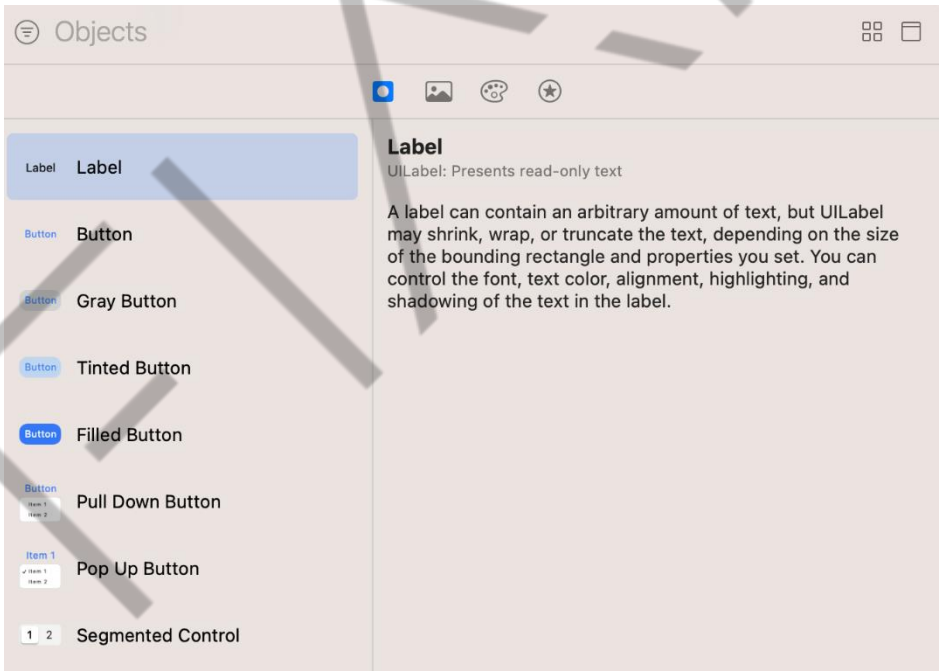


Figura 8 – Library – UI
Fonte: Elaborado pelo autor (2023)

Podemos acessar a Library utilizando a combinação de teclas SHIFT + COMMAND + L e para mantermos a Library sempre aberta podemos utilizar a combinação COMMAND + OPTION + SHIFT + L.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.2 Códigos e storyboard

1.2.1 Arquivos .swift

Em projetos iOS criados utilizando a linguagem *Swift,* nós armazenamos os nossos códigos-fonte em arquivos com extensão *.swift,* ou seja, sempre que você for criar uma classe, struct ou qualquer outra estrutura de código estará criando em arquivos *.swift.*

Nosso projeto vem com três arquivos com esta extensão. O primeiro arquivo chama-se **AppDelegate.swift**, e contém uma classe chamada **AppDelegate**, classe responsável por cuidar da inicialização do aplicativo e de tratar de toda a conversa realizada entre o sistema operacional (iOS) e o seu App.

Nesta classe existem métodos que serão chamados quando o aplicativo for inicializado, quando entrar em foco ou quando sair para background por exemplo, dentre outras coisas. É nesta classe que configuramos qualquer código que desejamos executar quando o App for inicializado, ou seja, qualquer preparação inicial do nosso App, que precise ser feita via código, pode ser colocada aqui.

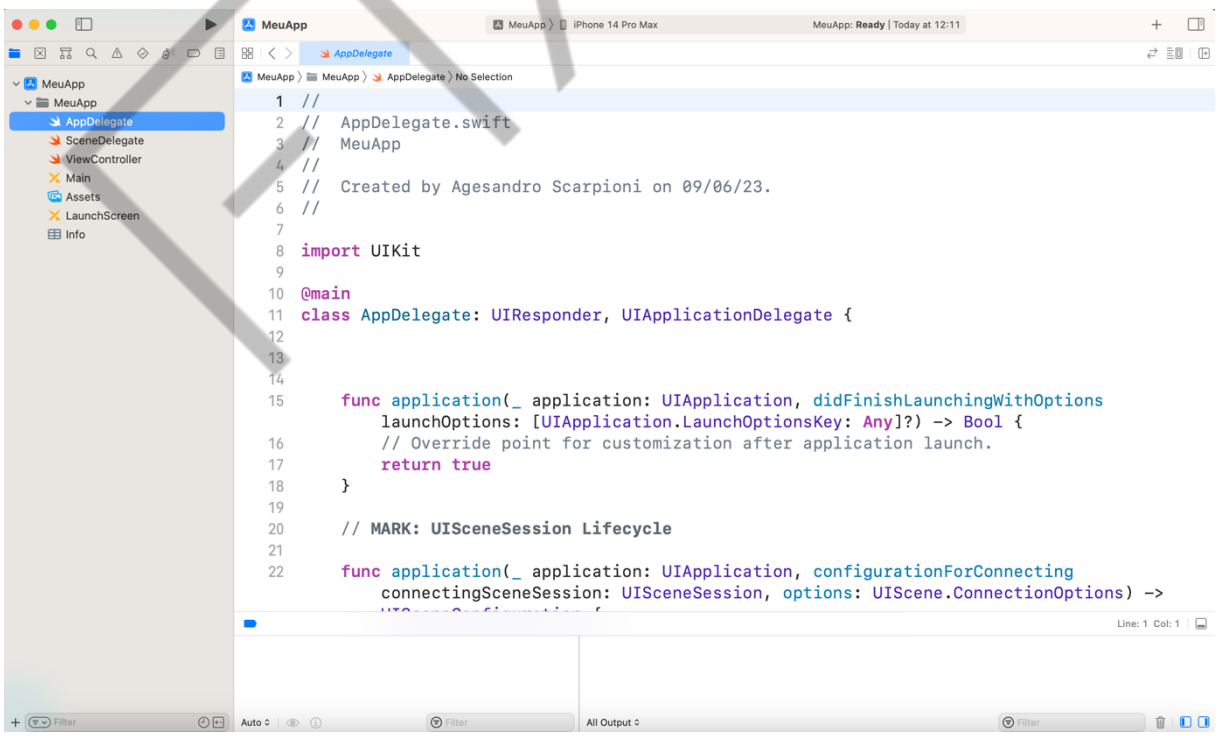


Figura 9 – Arquivo AppDelegate.swift
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A partir do iOS 13 foi incluso o arquivo **SceneDelegate.swift.** Com ele veio a possibilidade de trabalhar com múltiplas cenas dentro de um aplicativo, ou seja, para versões anteriores do iOS 13, um aplicativo tinha somente um processo e somente uma interface responsável por esse processo. No entanto, os aplicativos com iOS 13 e versões posteriores, podemos usar múltiplas cenas dentro de um aplicativo.

Para começar a trabalhar com nosso arquivo SceneDelegate é um muito simples, o nosso App Delegate tem diferentes funções, que deixam nossa aplicação saber de certos eventos, por exemplo quando o aplicativo foi para background e quando voltou de background. Essas funções foram mapeadas e agora serão feitas no nosso arquivo SceneDelegate, isso quer dizer, se seu aplicativo tiver duas cenas como mostrado na Figura “Um aplicativo com duas cenas” os eventos dessas telas serão informados de forma independente no SceneDelegate.

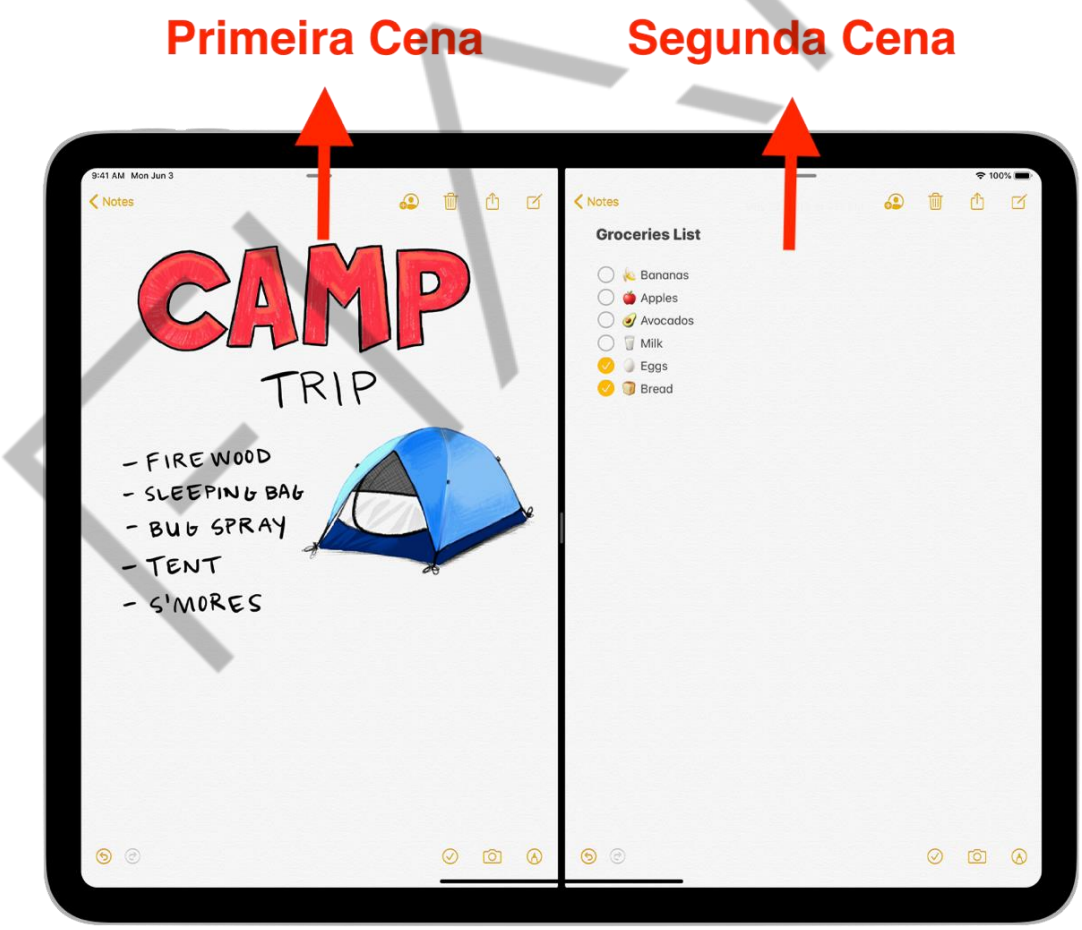


Figura 10 – Arquivo AppDelegate.swift
Fonte: Apple Developer (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora entraremos no arquivo SceneDelegate.swift. Veremos que os métodos que tínhamos no AppDelegate, como por exemplo o método application:willEnterForeground, agora podem ser implementados no nosso SceneDelegate como o scene:willEnterForeground.

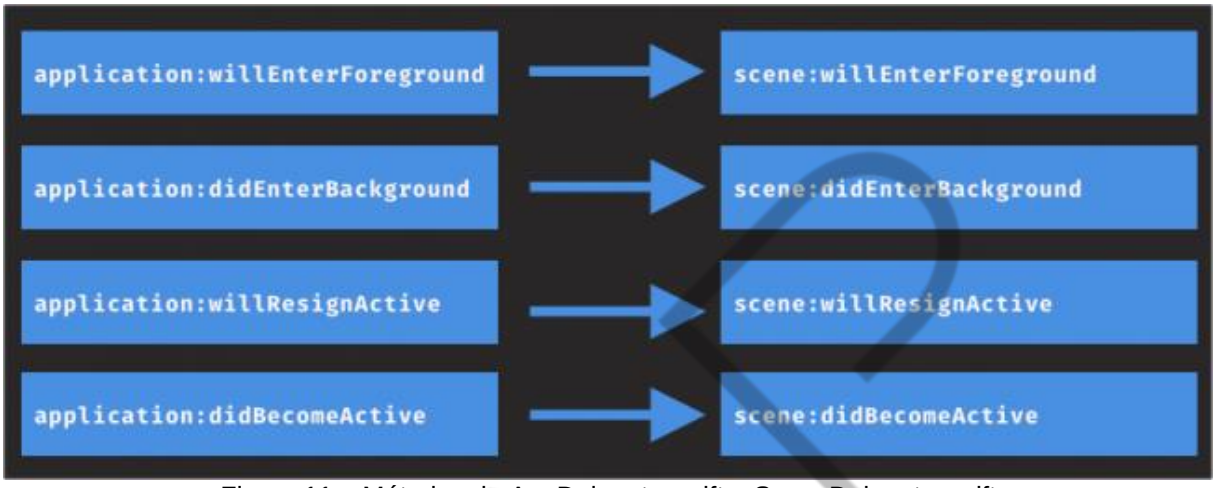


Figura 11 – Métodos do AppDelegate.swift e SceneDelegate.swift
Fonte: WWDC (2020)

Caso optarmos por dar suporte às versões antes do iOS 13, teremos que modificar algumas coisas. Primeiramente vamos falar que nosso App vai suportar a versão 12 do iOS. Para isso vamos clicar no nosso projeto, depois selecionaremos a aba *General* e então na sessão *Minimum Deployment*, selecionaremos o target iOS 12.0 como vemos na imagem.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

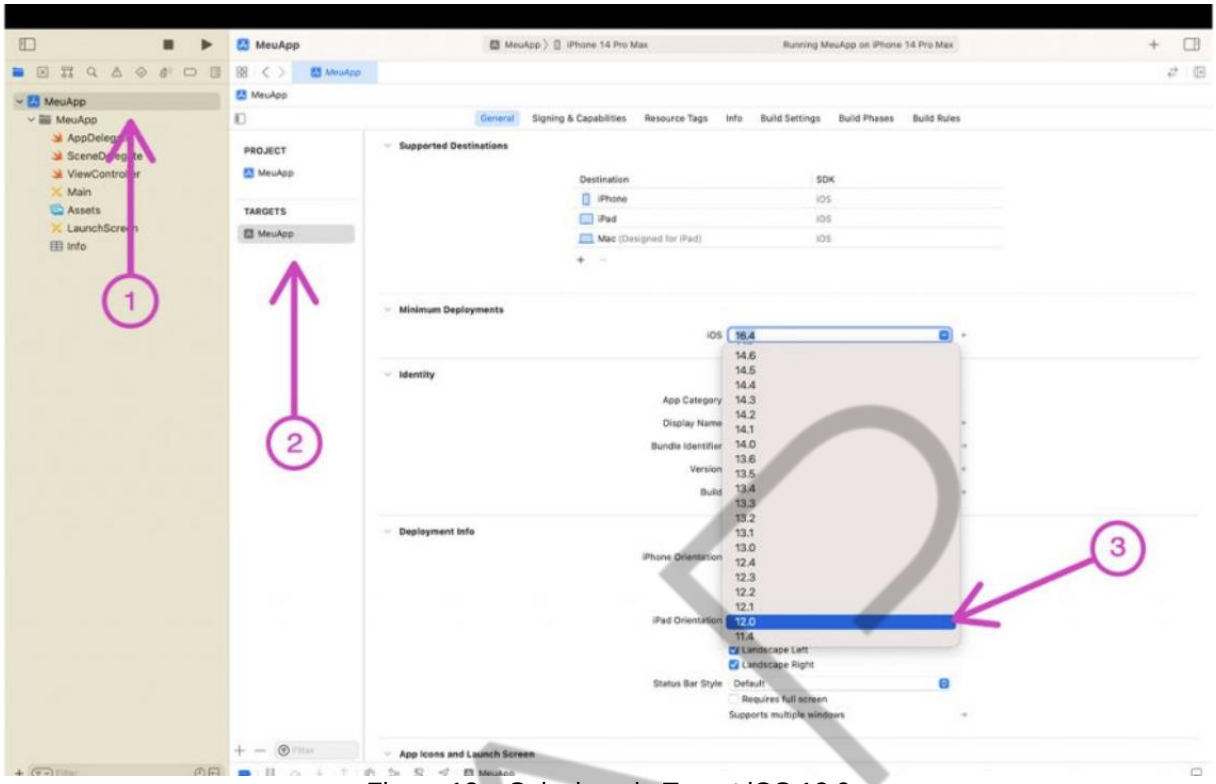


Figura 12 – Selecionado Target iOS 12.0
Fonte: Elaborado pelo autor (2023)

Note que se tentarmos rodar o app, mostrará alguns erros, pois estamos tentando usar funções que só são disponíveis no iOS 13 para cima. Para arrumar isso, vamos no arquivo SceneDelegate.swift e vamos incluir depois do import e antes do class o comando @available(iOS 13.0, *) ou seja, essa classe somente será disponível para iOS 13 e versões posteriores.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

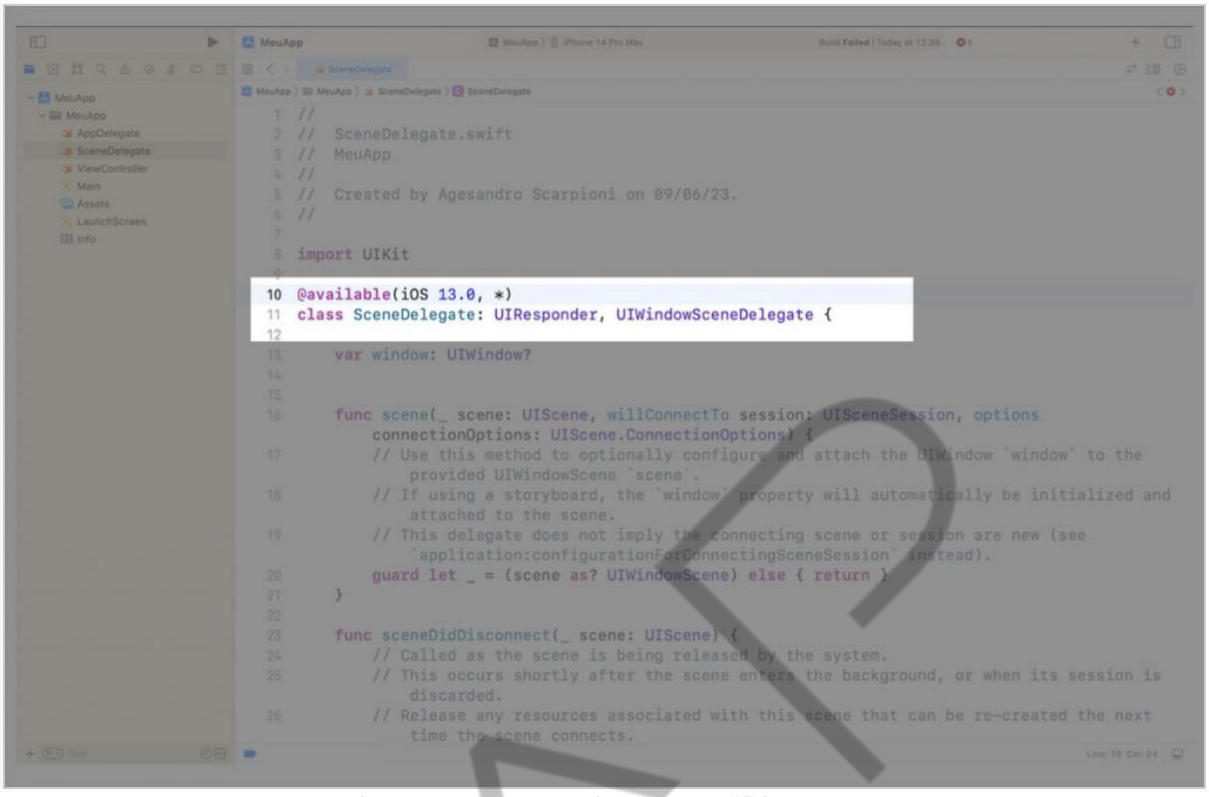


Figura 13 – SceneDelegate versão iOS 12.0
Fonte: Elaborado pelo autor (2023)

Dentro do nosso AppDelegate.swift, temos dois métodos que também precisam adicionar nosso @available(iOS 13.0, *), são eles o configurationForConnecting e o didDiscardSceneSessions.

O último passo para termos um aplicativo para suportar iOS 12 e anteriores é adicionar a UIWindow no nosso AppDelegate.swift, com o comando var window: UIWindow?. Sendo assim, nosso AppDelegate, ficará dessa forma:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

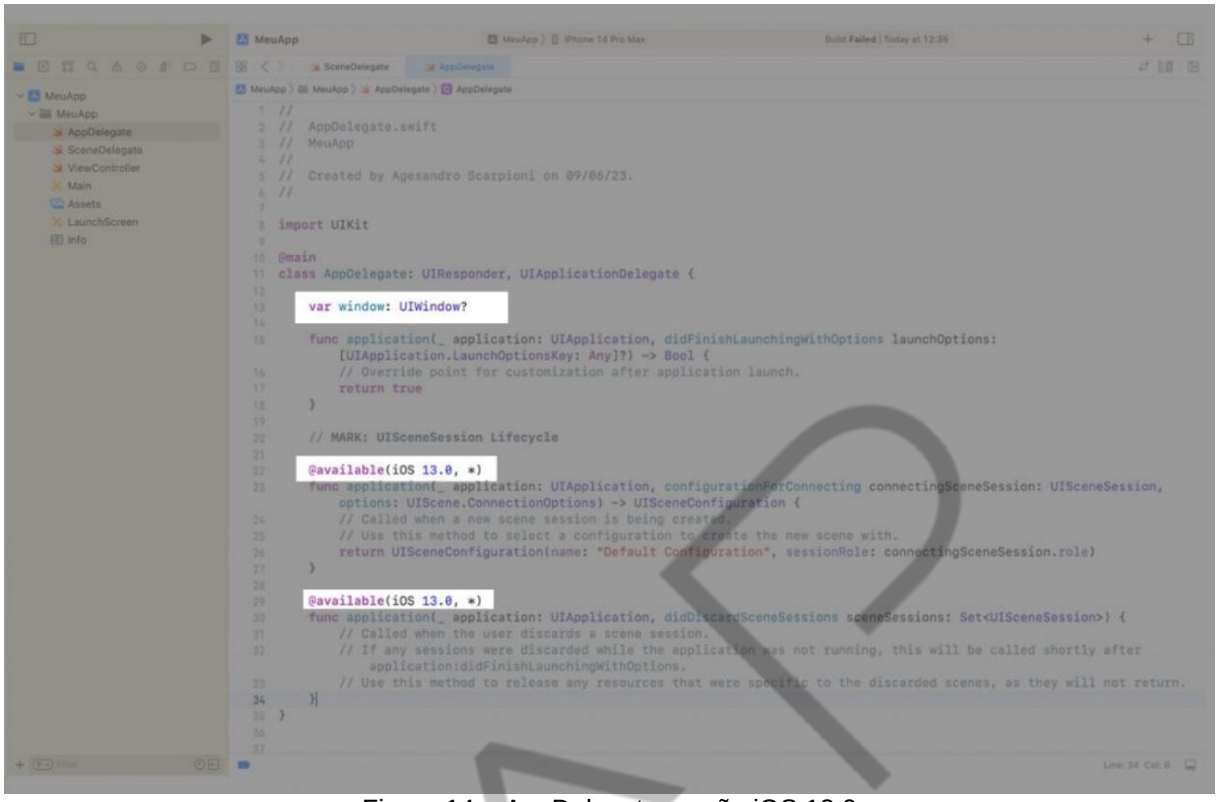


Figura 14 – AppDelegate versão iOS 12.0
Fonte: Elaborado pelo autor (2023)

Outro arquivo *.swift* que temos em nosso projeto é o **ViewController.swift**. Este arquivo contém uma classe chamada ViewController, que herda de uma classe chamada UIViewController, responsável por interagir com a tela inicial do seu App. Tanto a tela quanto a classe foram criadas pelo Xcode, porém, durante o desenvolvimento de um App, podemos criar outras telas e consequentemente outras classes para comandar essas telas. Em outro módulo falaremos mais sobre classes do tipo UIViewController e suas funcionalidades, o importante agora é entender que todos os seus códigos estarão presentes em arquivos *.swift.*

Vamos ver como criamos novos arquivos *.swift* em nosso projeto. Vá até o menu do Xcode e escolha File -> New -> File... (ou utilize o atalho COMMAND+N). Na tela seguinte, na área Source, podemos escolher que tipo de arquivo de código-fonte desejamos criar. Em se tratando de código-fonte que conterá classes etc. criadas em *Swift*, temos basicamente duas opções, **Cocoa Touch Class** ou **Swift File**. Utilizamos Cocoa Touch Class quando desejamos criar uma classe que herdará alguma outra classe presente nos frameworks existentes em iOS. Por exemplo, vimos que temos em nosso projeto a classe ViewController (presente no arquivo ViewController.swift) e esta classe herda de UIViewController. Se estivéssemos querendo criar uma outra

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

classe do tipo UIViewController, poderíamos escolher a opção Cocoa Touch class, pois desse modo a tela a seguir nos traria a opção de definir de que classe queríamos herdar. Nos casos onde estamos criando nossas próprias classes, que não herdam de nenhuma outra, podemos simplesmente escolher *Swift* file, que é o que vamos escolher agora. Na tela seguinte, selecionamos o nome do arquivo (chame de **MyClass**) e clique em **Create**.

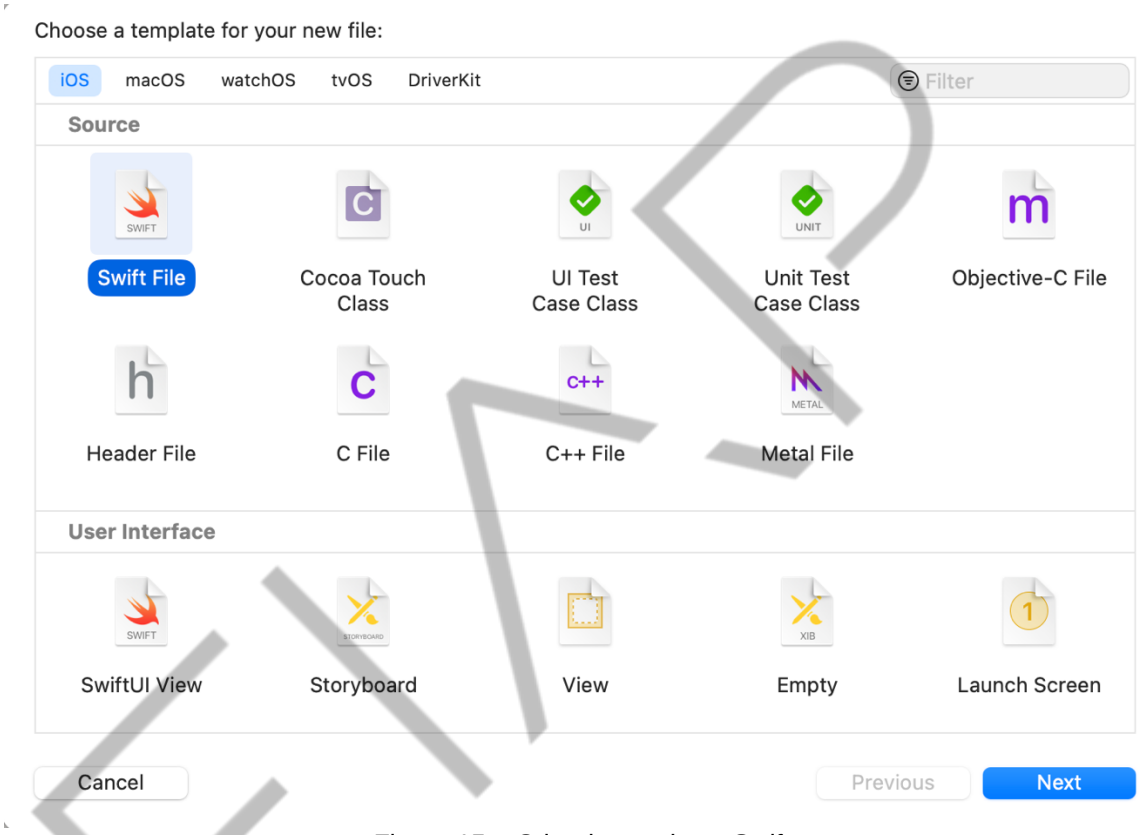


Figura 15 – Criando arquivos. Swift
Fonte: Elaborado pelo autor (2023)

Teremos o arquivo MyClass.swift onde criaremos nossa própria classe e utilizá-la em qualquer outro lugar do App.

1.2.1.1 Controle de acesso

Interessante saber que em *Swift* o controle de acesso às classes, propriedades e métodos são feitos baseado em módulos e arquivos, e por padrão todas as classes que criamos são marcadas como internal, que indica que aquela classe pode ser acessada a partir de qualquer outra classe dentro do seu projeto.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em *Swift,* um módulo representa uma unidade de código que pode ser distribuído. Um App ou um framework são tratados como módulos. Um pedaço de código pode estar em um simples arquivo dentro do seu App ou fazer parte de um módulo (framework) separado.

Ao todo temos cinco tipos de controles de acesso em *Swift.* **Open** ou **Public** permitem que o código possa ser utilizado dentro do seu módulo e em outros módulos que o importem. Geralmente usamos **Public** ou **Open** quando precisamos criar código que será distribuído a terceiros. Internal permite que aquele código seja utilizado dentro do módulo que foi definido, e é o tipo utilizado quando criamos uma classe e não definimos explicitamente seu controle de acesso (como temos feito até aqui no curso). *Fileprivate* define que aquele código será visível somente dentro do seu próprio arquivo, e *Private* apenas dentro do próprio bloco que foi definido. No arquivo *MyClass* recém-criado, vamos desenvolver uma nova classe conforme abaixo:

```
import Foundation

class MyClass {

    private var name: String

    func getWelcomeMessage() -> String {
        return "Esta foi a primeira classe criada por \(name)
em um projeto iOS"
    }
    init(name: String) {
        self.name = name
    }
}
```

Código-fonte 1 – Classe MyClass
Fonte: Elaborado pelo autor (2023)

Como não definimos explicitamente o controle de acesso da nossa classe, foi definida como internal e sendo assim, está visível em qualquer parte do nosso código. Seu método *getWelcomeMessage()* também pode ser chamado normalmente porém como a propriedade *name* foi definida como *private* pode ser acessada dentro da própria classe, ou seja, se tentarmos recuperar esta informação fora da classe não poderemos.

Vamos ver isto na prática. Abra o arquivo ViewController.swift e, substitua o método viewDidLoad pelo abaixo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
override func viewDidLoad() {
        super.viewDidLoad()

        let myClass = MyClass(name: "Agesandro Scarpioni")
        //print(myClass.name) // Este código não funcionará
se for descomentado por conta do private
        let message = myClass.getWelcomeMessage()
        print(message)
}
```

Código-fonte 2 – Acessando a classe MyClass
Fonte: Elaborado pelo autor (2023)

O método *viewDidLoad* é executado sempre que uma tela é carregada, e o que fizemos foi simplesmente criar um objeto do tipo *MyClass,* recuperar o retorno do método *getWelcomeMessage* atribuindo a uma variável e imprimindo esta variável no console. Note que não conseguimos acessar a propriedade *name* pois é *private* e sendo assim pode ser acessada dentro da própria classe (o método *getWelcomeMessage* utiliza ela).

Clique no botão *Run* para que o aplicativo seja compilado e executado no simulador. Neste momento, aparecerá o simulador representando o iPhone 14 Pro Max (que foi selecionamos anteriormente no início do projeto) e, após o simulador terminar de carregar o iOS, o App será executado, mostrando uma tela em branco e, na *Debug Area,* mostrando a mensagem retornada pelo método.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

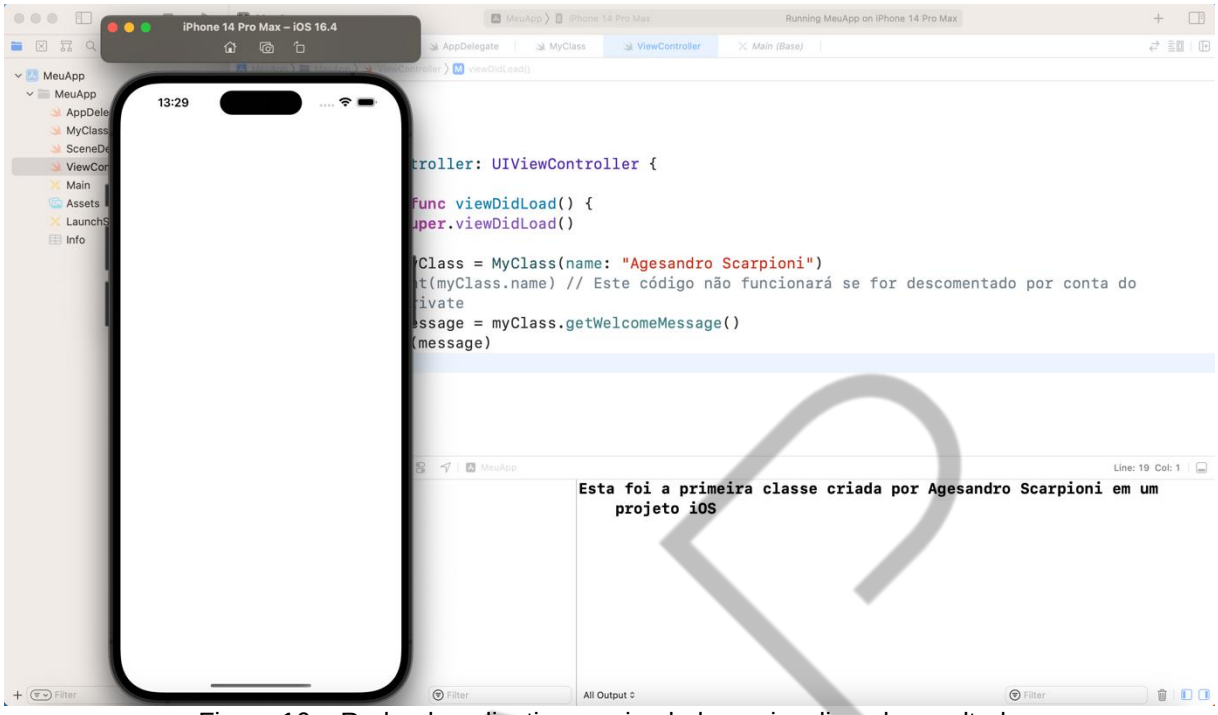


Figura 16 – Rodando aplicativo no simulador e visualizando resultado
Fonte: Elaborado pelo autor (2023)

1.2.2 Storyboard

Agora que sabemos como trabalhar com o código-fonte de um App, vamos entender como podemos manipular as telas dos nossos aplicativos. Sempre que iniciamos um novo projeto, são fornecidos dois arquivos com a extensão .storyboard são eles o Main e o LaunchScreen.

Storyboard é um ambiente criado pela Apple onde podemos ter uma visão de todas as telas que nosso App possui e, além disso, como estas telas estão relacionadas e interconectadas. A ideia é que o desenvolvedor tenha uma visão do seu App da mesma maneira que um diretor de cinema consegue visualizar como as cenas do seu filme deverão ser filmadas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

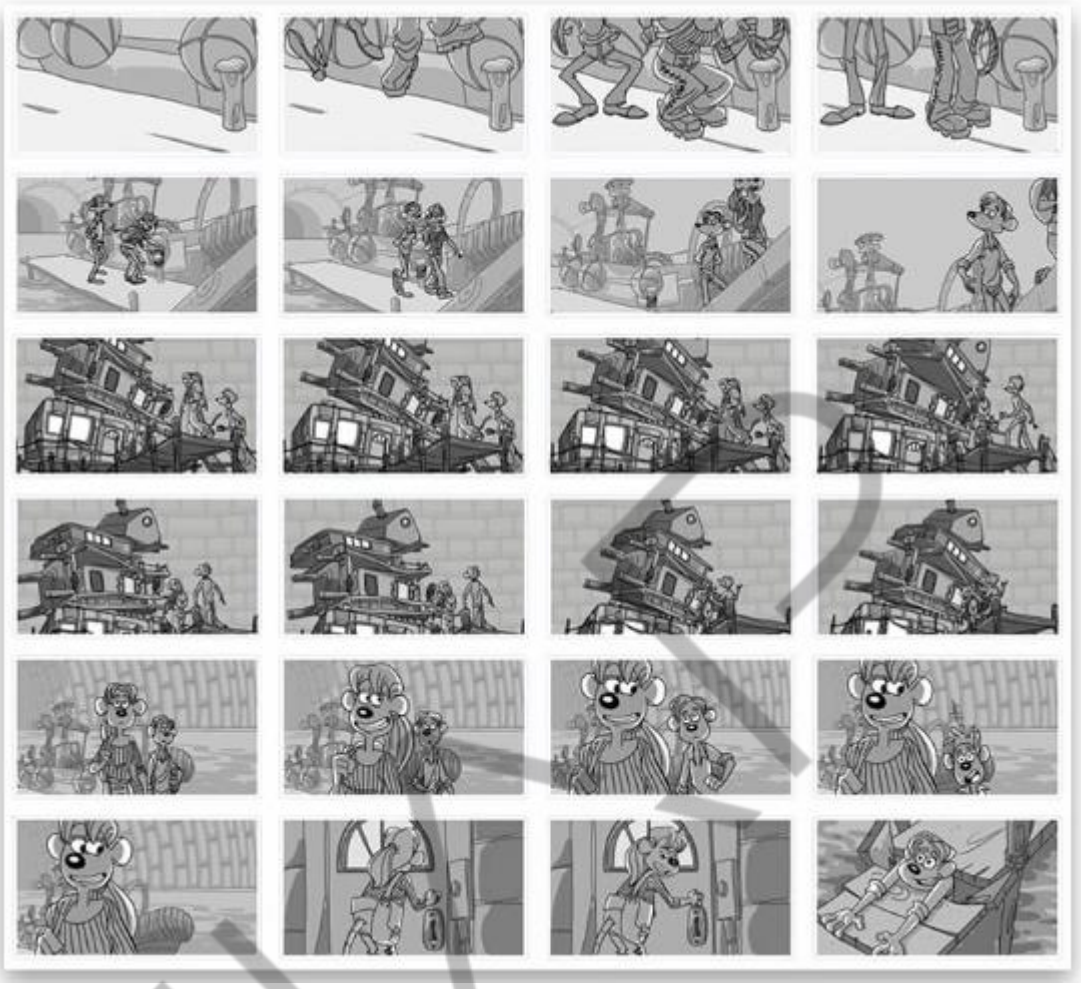


Figura 17 – Exemplo de um *storyboard* de um filme
Fonte: Livlily (2011)

O arquivo *LaunchScreen.storyboard* contém a tela que aparecerá enquanto o aplicativo está sendo carregado e o *Main.storyboard* é onde as telas do nosso App serão configuradas. Abra o arquivo *Main.storyboard* e perceba que temos uma representação da tela inicial do nosso aplicativo que, por enquanto, é apenas uma tela em branco. No *Storyboard,* uma tela é chamada de *Scene* (cena) e precisa estar vinculada a uma classe que herde de *UIViewController* para que seus componentes possam ser manipulados via código e que as ações que o usuário faz na tela possam ser interpretadas. Como comentado anteriormente, ao selecionarmos um *storyboard* nossa Editor Area passa a se chamar *Interface Builder.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

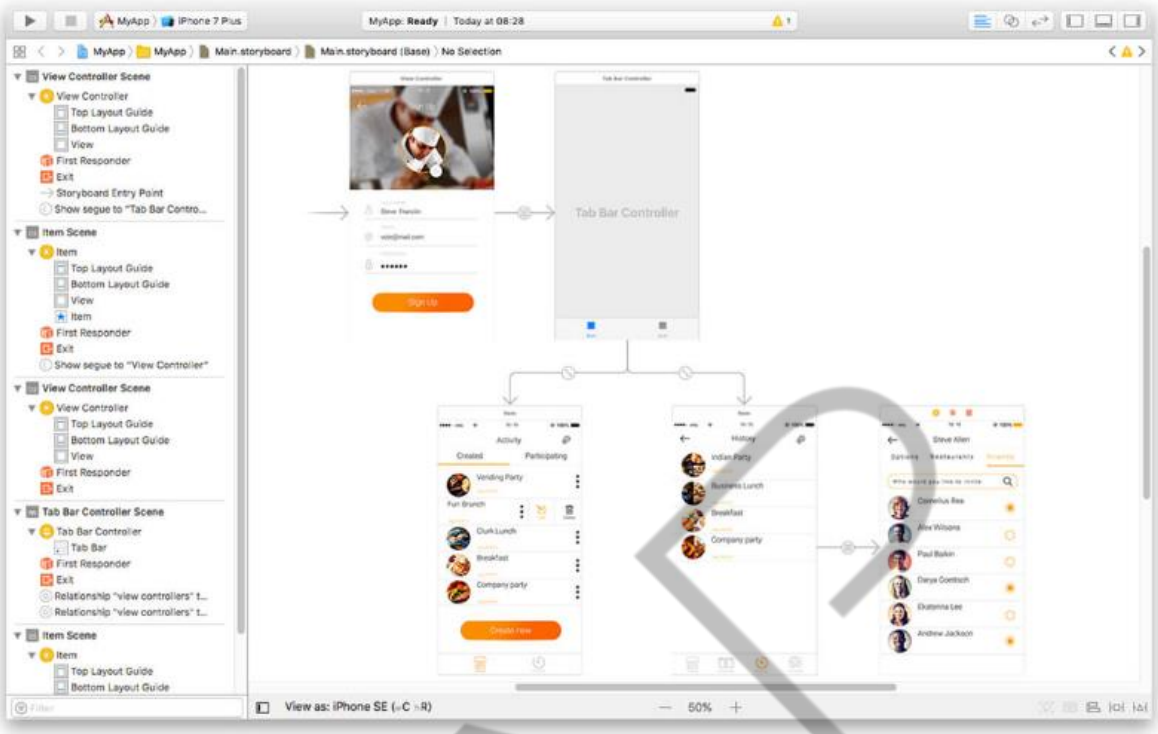


Figura 18 – Exemplo de *storyboard*
Fonte: Elaborado pelo autor (2018)

Nosso aplicativo possui apenas uma tela e consequentemente apenas uma cena, porém é comum que o *storyboard* contenha várias cenas representando as várias telas de um App. Também é possível dividir um aplicativo em vários *storyboards*, pois quanto mais cheio de cenas tiver, mais pesado fica ao ser manipulado no *Interface Builder.*

Quando um *storyboard* contém muitas cenas, às vezes é necessário modificar o nível de zoom de visualização delas. Isso é feito através dos botões + e – presente na sua parte inferior. Outra característica interessante é que cada cena é representada como se estivesse em um determinado *device* e em determinada orientação (*landscape, portrait*), e é possível visualizar como aquela cena ficaria em outro *device* ou orientação clicando no botão que diz *View* as: iPhone 8. Ao clicar neste botão, aparece um painel onde podemos escolher em que *device* queremos visualizar nossas cenas e em qual orientação.

Cada componente inserido dentro de uma tela é visualizado através de uma lista presente no lado esquerdo do *Interface Builder.* Esta lista chama-se **Document Outline.** Nela vemos como nossos componentes estão apresentados, por exemplo, qual está mais para trás, mais para frente, que componente está dentro de outro etc.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No nosso exemplo, o único componente presente na cena é a **View** principal, o componente onde todos os demais ficam inseridos.

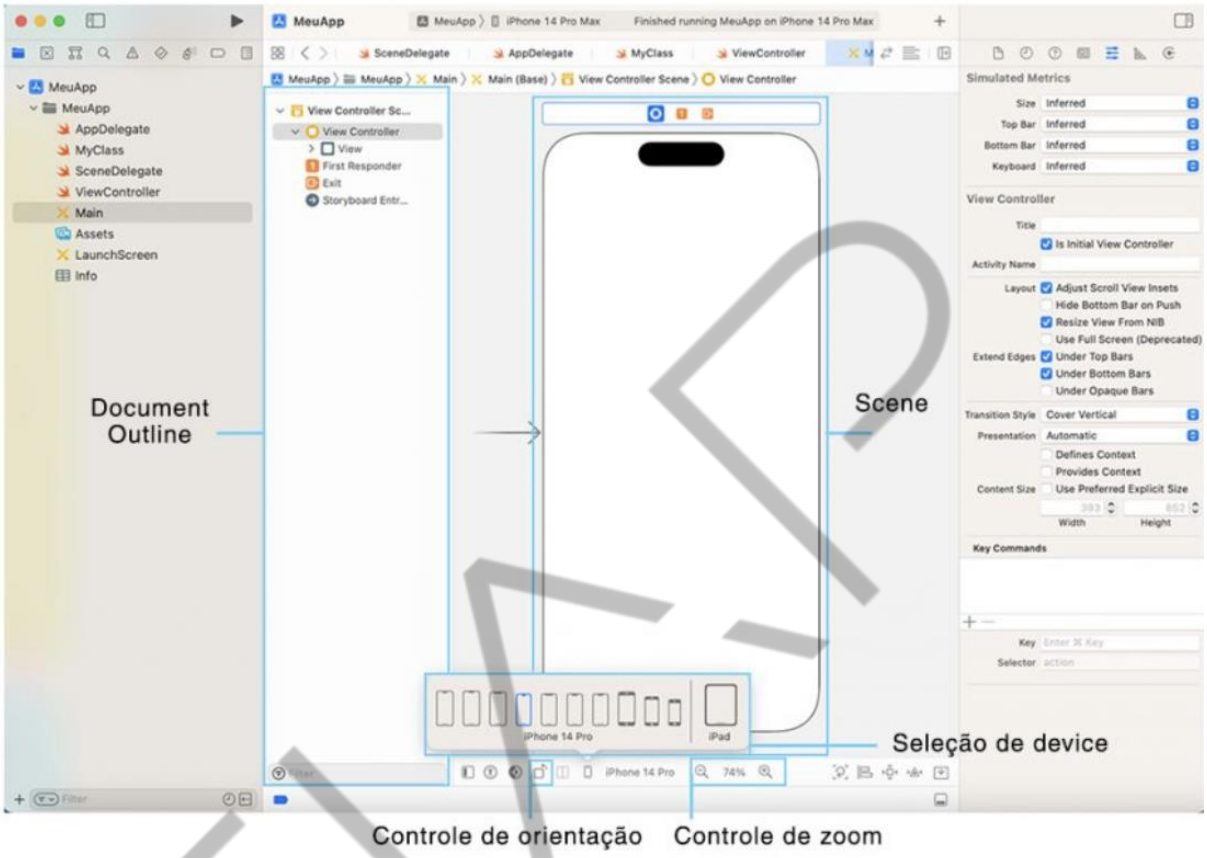


Figura 19 – Interface do *Storyboard*
Fonte: Elaborado pelo autor (2023)

1.3 Criando conta developer

Para que possamos executar nosso App em um *device,* ao invés de um simulador, precisamos configurar no Xcode uma conta de desenvolvedor. Existem dois tipos de contas, *a paga e a gratuita.* Felizmente não precisamos de uma conta paga para executarmos os aplicativos em *devices* reais, porém é necessária para que o desenvolvedor possa publicar seu aplicativo na App Store, além de poder testar outros recursos como por exemplo o uso de Push Notifications. Vamos aprender como criar nossa conta (gratuita) e como configurá-la no Xcode.

Primeiramente, precisamos acessar o site de desenvolvedores da Apple, através do link <https://developer.apple.com/>. Ao entrar no site, clique em **Account**, que se encontra no canto superior direito da página.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

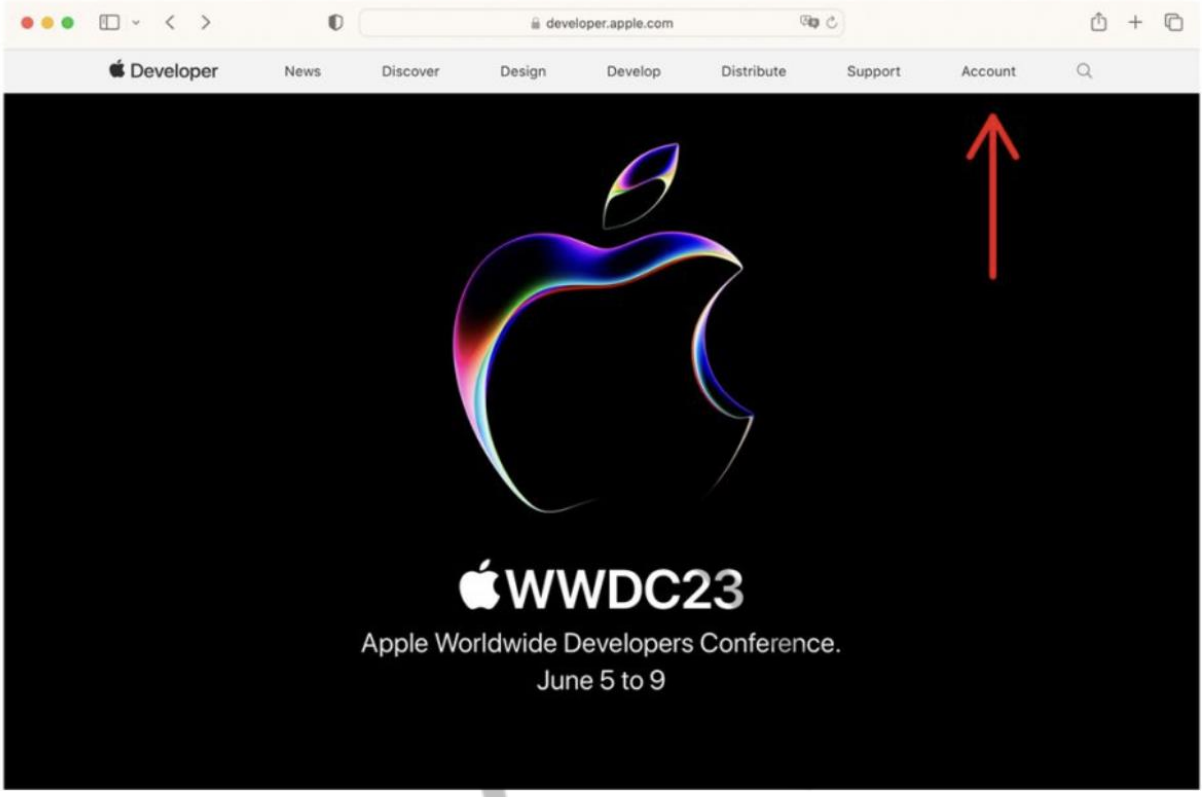


Figura 20 – Site de developers da Apple
Fonte: Elaborado pelo autor (2023)

Nesta página podemos criar uma conta de desenvolvedor entrando com o seu Apple ID. Caso não possua uma conta Apple, podemos utilizar essa mesma página para criá-lo, basta seguir as instruções. Caso possua um Apple ID (se você possui qualquer dispositivo da Apple provavelmente tem um Apple ID que é o e-mail que usou na sua conta Apple) basta logar com as suas credenciais e acessar a área de desenvolvedores.

Para criar um Apple ID, basta preencher todas as informações solicitadas, como primeiro e último nome, país, data de nascimento etc. Ao final da página, após preencher todas as informações, clique em **Continue**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

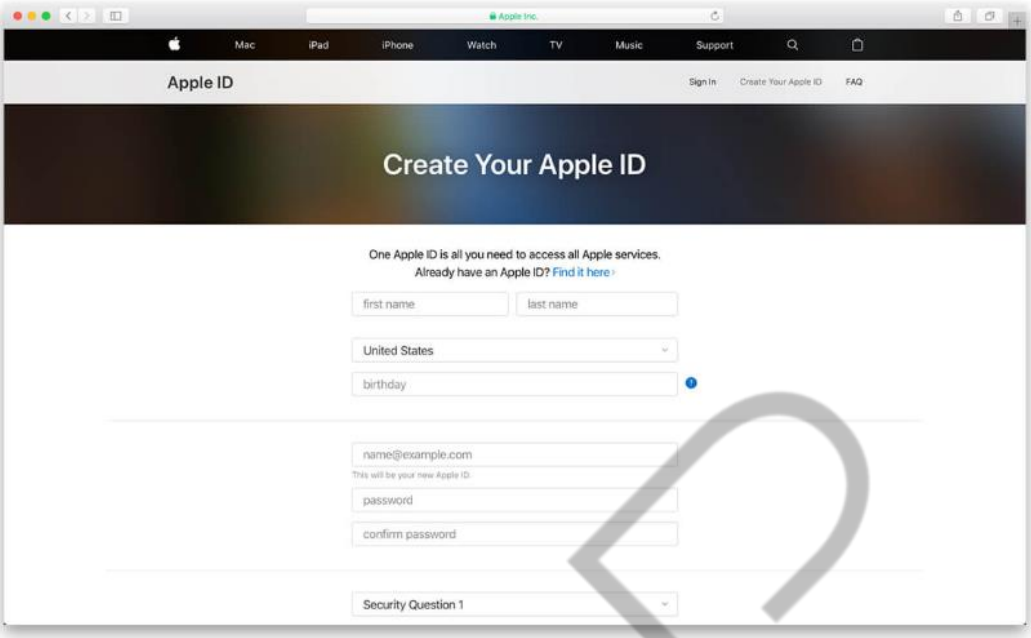


Figura 21 – Criando conta de desenvolvedor
Fonte: Elaborado pelo autor (2018)

Nesse momento você verá uma tela onde deverá preencher o código que foi enviado para o seu e-mail. Preenchendo o código, você terá finalizado o processo de criação da conta, e estará logado na área de desenvolvedores.

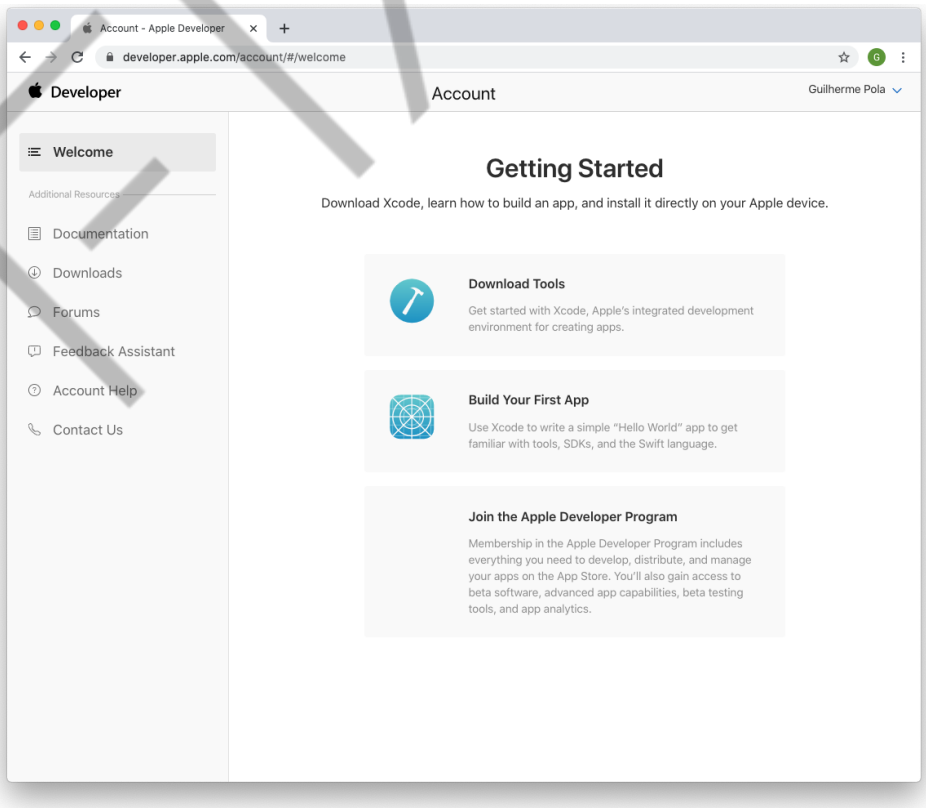


Figura 22 – Logado na área de desenvolvedor
Fonte: Elaborado pelo autor (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, com a conta criada, precisamos configurá-la no Xcode para que possamos rodar nosso pequeno App em nosso *device.*

1.4 Rodando Apps no *device*

Agora que temos nossa conta de desenvolvedor criada, precisamos configurá-la no Xcode e em nosso projeto. Lembra quando criamos nosso projeto, que deixamos o campo Team em branco? Este é o campo que precisamos preencher com os dados do desenvolvedor para que o projeto possa ser atribuído e vinculado à nossa conta e, dessa maneira, esteja apto tanto a rodar em nosso *device* quanto, futuramente, a ser publicado na App Store. Não se preocupe, este processo pode ser feito posteriormente, e é isso que faremos agora.

Primeiro, vamos configurar o Xcode para que tenha acesso à nossa recém-criada conta. Para isso, vá no menu **Xcode -> Settings...** (ou COMMAND+,) e selecione a segunda aba, **Accounts**. Nesta tela, clique no botão + (localizado no canto inferior esquerdo) e na tela que segue selecione Apple ID dentre as opções e clique em **Continue**.

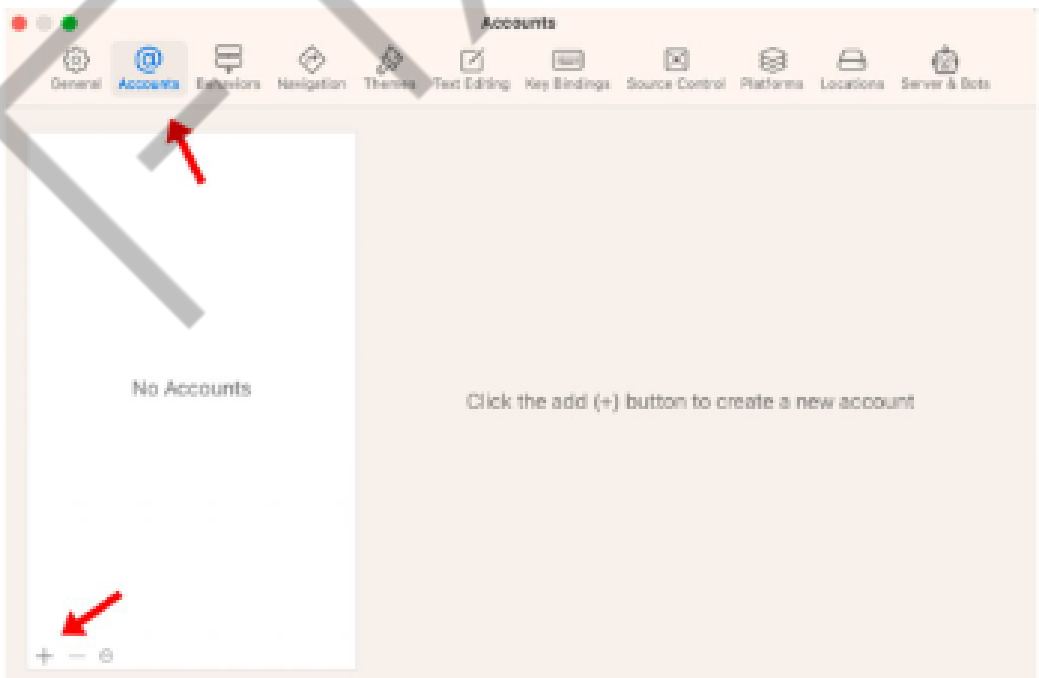


Figura 23 – Entrando em Accounts e Clicando no +
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

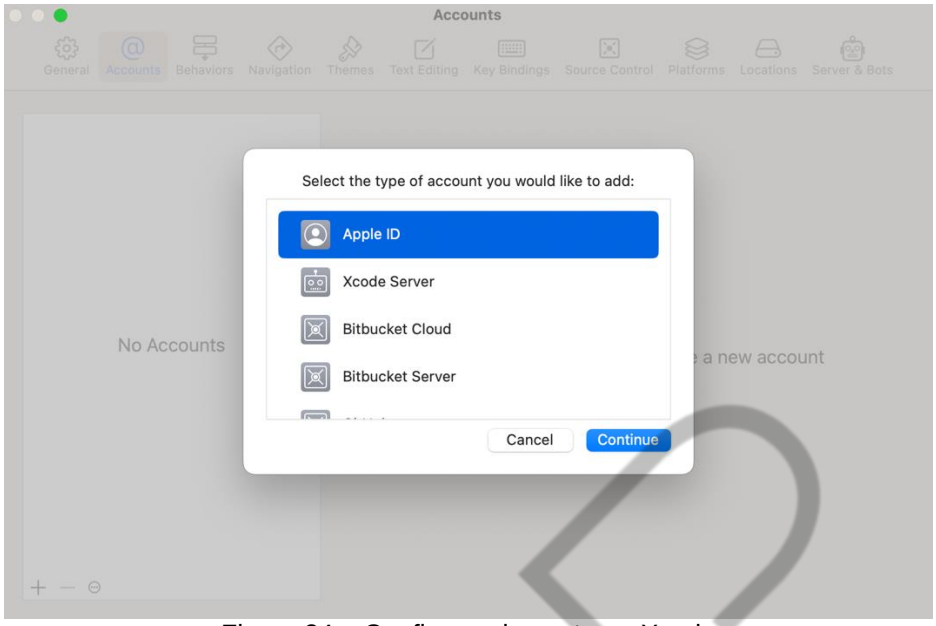


Figura 24 – Configurando conta no Xcode
Fonte: Elaborado pelo autor (2018)

Agora basta preencher o e-mail associado à sua conta, clicar em **Next** e logo depois preencher sua senha. Ao clicar em **Next** novamente, você terá configurado sua conta no Xcode.

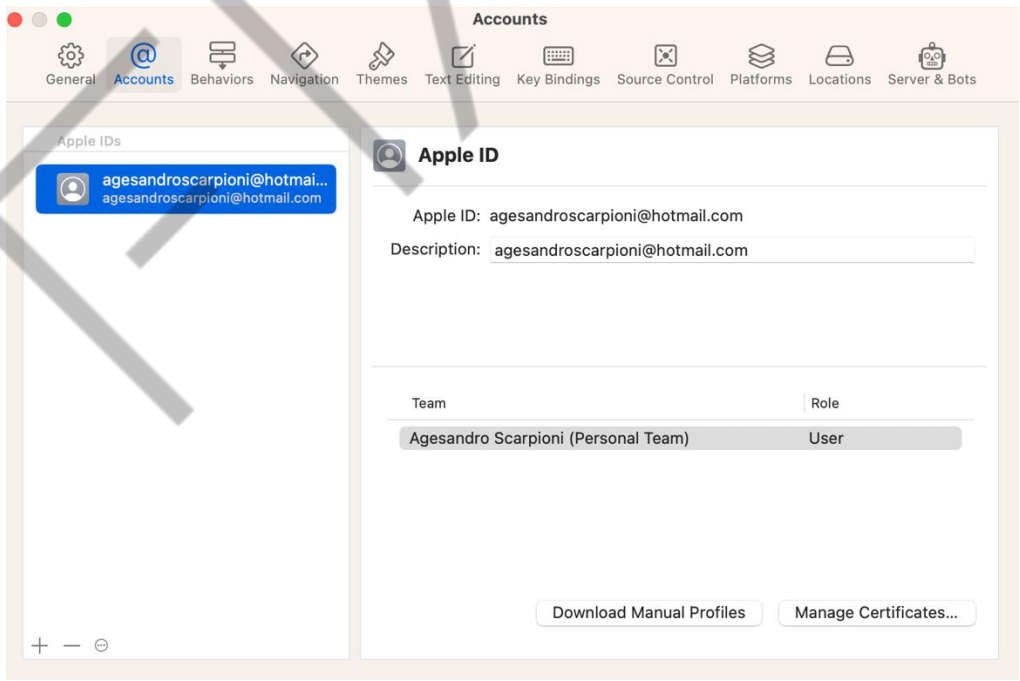


Figura 25 – Conta configurada
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esta etapa precisa ser feita somente uma vez. O vínculo criado foi com o Xcode e não com o seu projeto, ou seja, qualquer projeto que criar daqui para frente pode ser vinculado a esta sua conta, basta apontá-la na opção Team, no momento da criação do projeto. Para o nosso projeto atual, em que deixamos a opção Team como None. Vamos ao procedimento:

Na **Navigator Area**, com o **Project Navigator** selecionado, clique no seu projeto e na tela que abrir certifique-se de estar na aba **Signing & Capabilities**. Na opção Team, que está marcada como *None*, clique no menu *dropdown* e selecione a sua conta.

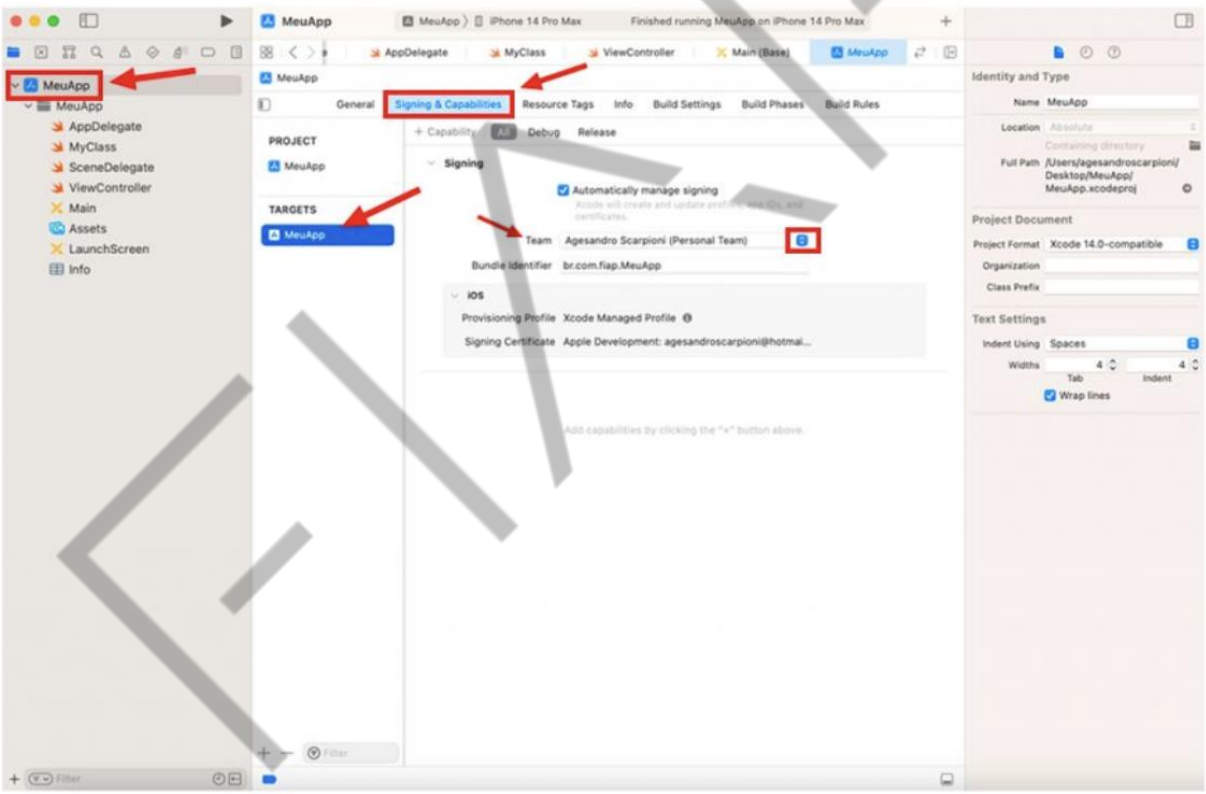


Figura 26 – Vinculando conta ao projeto
Fonte: Elaborado pelo autor (2020)

Pronto, o projeto está vinculado à sua conta de desenvolvedor e com isso você já pode plugar seu iPhone ou iPad e testar o aplicativo no seu *device*. Agora, plugue seu celular no Mac e, se surgir no *device* uma mensagem pedindo para confiar no computador, selecione **Confiar** (ou **Trust** caso seu iOS esteja em inglês) e aguarde enquanto o Xcode prepara seu iPhone para receber seus aplicativos. Este processo pode demorar um pouco.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Após algum tempo, se você clicar no **Run Destination** notará que seu iPhone aparecerá como um *device* válido para rodar o App. Selecione e clique no botão *Run*. Pode ser que você tenha que configurar algumas permissões no seu *device.* Caso apareça algum alerta, basta seguir as instruções (que devem ser feitas no seu aparelho) e, após tudo configurado, clique em *Run* novamente para ver seu App rodando no seu próprio *device.*

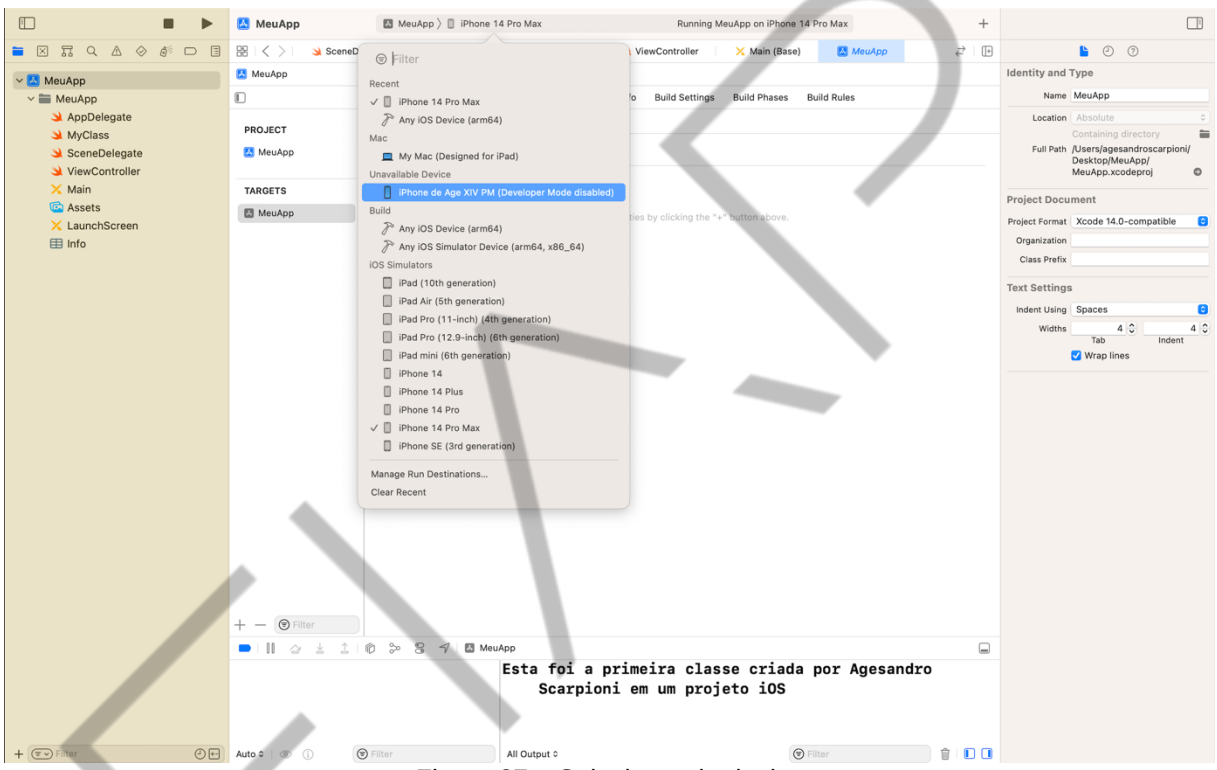


Figura 27 – Selecionando device
Fonte: Elaborado pelo autor (2023)

Agora podemos executar o aplicativo tanto nos simuladores disponíveis quanto em dispositivos iPhone ou iPad.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

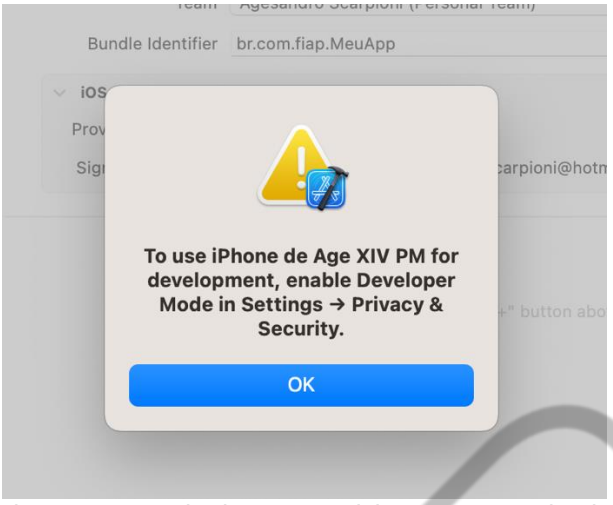


Figura 28 – Modo de Desenvolvimento - Desativado
Fonte: Elaborado pelo autor (2023)

Caso execute o App e apareça a mensagem que o seu iPhone está desabilitado no modo desenvolvedor, vá em ajustes do seu aparelho, clique em Privacidade e Segurança, procure Modo de Desenvolvimento e o ative, uma mensagem sobre segurança aparece e depois clique em Reiniciar.

Com o telefone desbloqueado rode novamente seu aplicativo, possivelmente irá aparecer uma outra mensagem de “Desenvolvedor Não Confiável”, clique em cancelar, vá em “Ajustes” no seu Iphone, escolha a opção “Gestão de VPN e Dispositivo”, clique no ítem “App do Desenvolvedor (não confiável)”, depois na próxima tela escolha “Confiar em “Apple Development: xxx@xxxx.com” e por último clique no botão vermelho “Confiar”, pronto, na próxima vez que clicar em start seu App irá aparecer no seu dispositivo.

1.5 Criação do App Hello Universe

Para finalizar este módulo, vamos criar um simples aplicativo. Sabemos que os deuses da Programação exigem que, para que um desenvolvedor tenha sucesso, crie um Hello World como seu primeiro projeto. Como queremos ser desenvolvedores bem-sucedidos, iremos um pouco além e, ao invés de criarmos um *Hello World* vamos criar um *HelloUniverse*, afinal, para quê dar alô para o mundo quando podemos dar alô para o universo inteiro, certo? Então, mãos à obra!

A partir de agora faremos uso de alguns componentes sem necessariamente entendermos 100% sobre seu uso. A ideia é apenas termos um App dizendo *Hello*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Universe e passarmos pela aprovação dos Deuses da Programação, portanto, segure um pouco a emoção pois cada componente usado terá seu devido momento de explicação aqui no curso, tudo bem?

Crie um novo projeto, chamado **HelloUniverse** e com os mesmos dados usados no aplicativo anterior (iOS, App, FIAP, br.com.fiap etc.). Aproveite e vincule sua conta na opção Team.

Com o projeto criado, abra o **Main.storyboard** pois adicionaremos alguns componentes na nossa cena. Primeiro que estamos saudando o universo, vamos trocar a cor branca da nossa tela por uma cor mais escura, um azul bem escuro. Para isso, clique no meio da cena, selecionando assim a **View**, e no **Utilities Panel** clique no inspector que se assemelha a um painel equalizador de som, (quinto botão no canto superior direito). Este é o **Attributes Inspector**, e é onde configuramos os atributos do componente selecionado dentro de uma cena. Como selecionamos a View, estamos neste momento mudando os seus atributos, as suas propriedades. A propriedade que mudaremos é a **Background**, que define qual a cor de fundo da nossa tela. Clique na área branca ao lado e com isso o painel de cores será aberto. Selecione a aba **Color Sliders** e defina como cor os valores 1 para Red, 20 para Green e 58 para Blue. Os passos estão definidos na imagem abaixo.

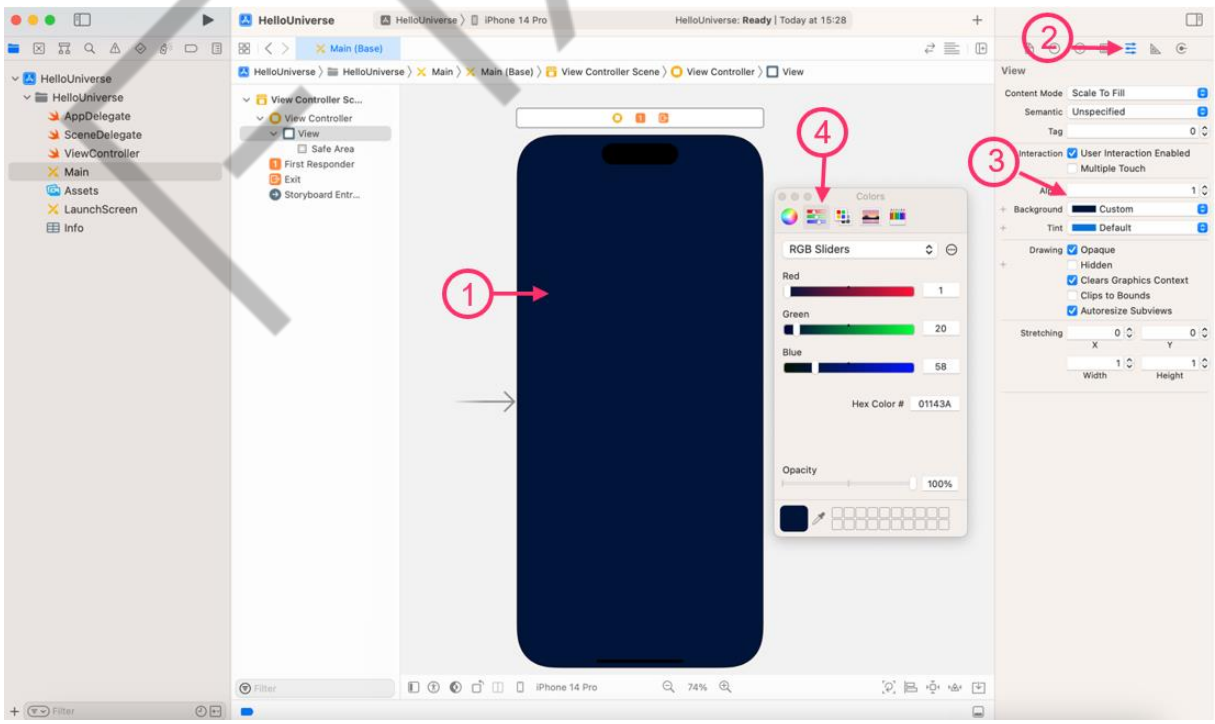


Figura 29 – Configurando a tela inicial do app
Fonte: Elaborado pelo autor (2020)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, vamos adicionar um Label ao nosso App, pois é o componente utilizado quando precisamos inserir informações textuais na tela. Assim como todos os demais componentes, o Label encontra-se na Library, para acessar a Library podemos clicar no botão “+” no canto direito superior ou então podemos abrir com a tecla de atalho COMMAND + SHIFT + L. Na Library existem vários componentes, dentre eles o Label, porém uma forma mais simples de selecionar o componente desejado é filtrando a lista por nome. Clique no campo logo acima dessa área e digite Lab. Isso fará com que a lista seja filtrada mostrando apenas o componente Label. Arraste este componente para o meio da tela e após isto aplique dois cliques no mesmo. Isso possibilita editar o texto presente no Label. Digite Hello Universe e pressione ENTER.

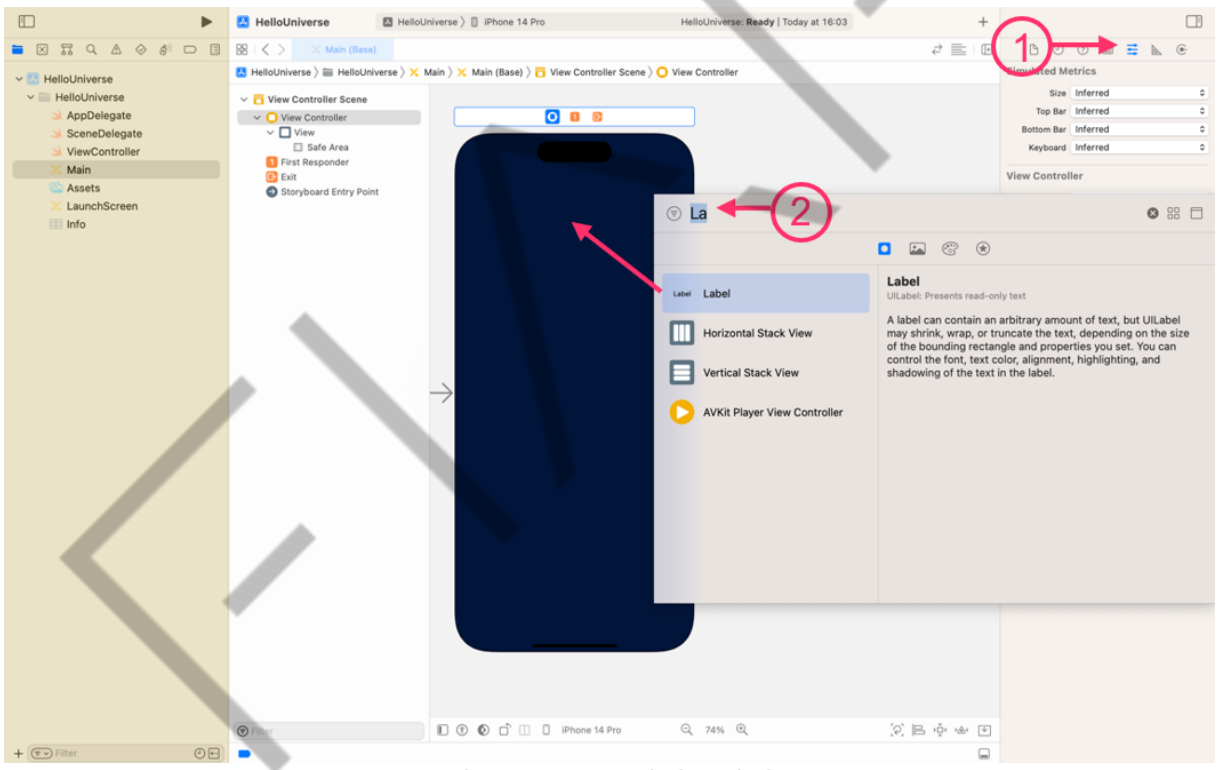


Figura 30 – Inserindo Label
Fonte: Elaborado pelo autor (2023)

Nosso Label está com a cor preta, o que não combina muito bem com a cor escura de fundo da tela, então devemos mudar para uma cor mais clara como por exemplo o amarelo. Selecionado, vá ao Attributes Inspector e troque a propriedade Color (que define a cor do texto) para uma cor mais amarelada. Aproveitando, vamos mudar também o tamanho e o tipo de fonte associados ao nosso Label. Logo abaixo da cor, temos a propriedade Font, onde podemos selecionar o tipo de fonte, tamanho

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

e estilo (negrito, itálico etc.). Para o nosso exemplo, vamos escolher o tamanho 36 e o estilo Light e em Alignment escolha centralizado.

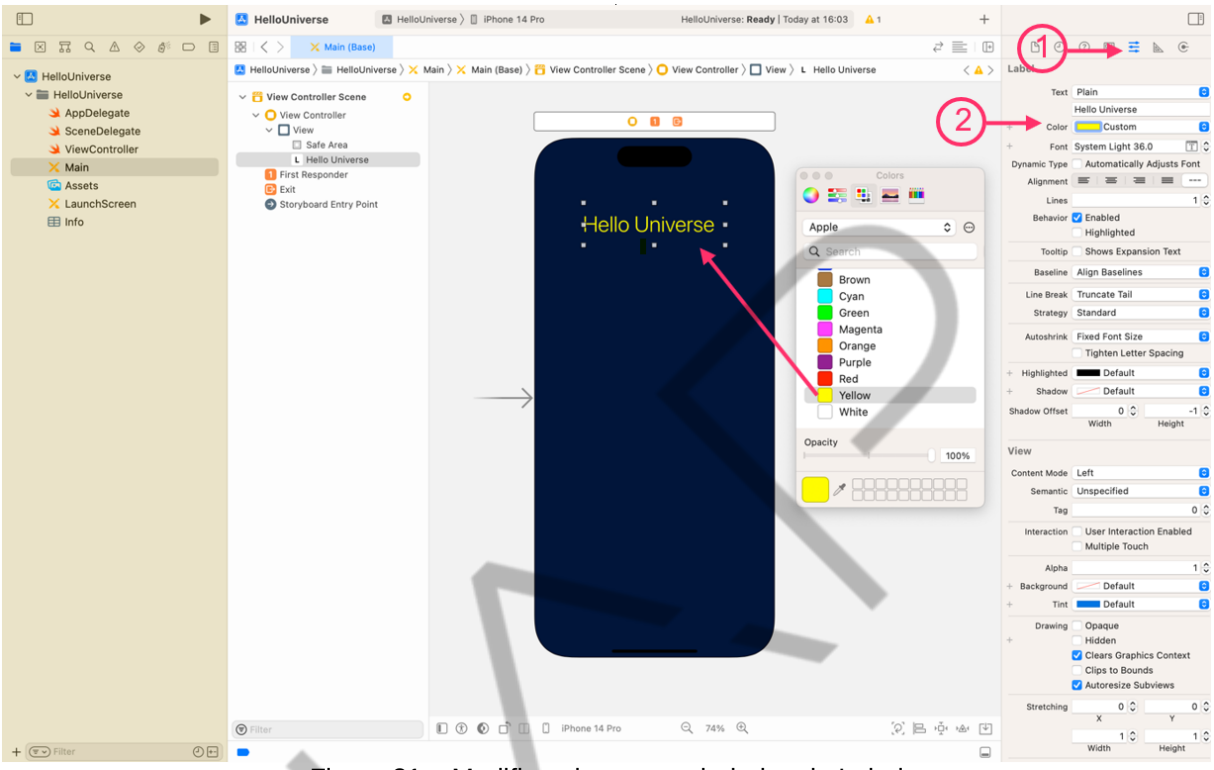


Figura 31 – Modificando as propriedades do Label
Fonte: Elaborado pelo autor (2023)

Para finalizar, vamos adicionar uma imagem do universo ao nosso App. Pesquise no Google uma imagem que represente um universo (no meu caso, utilizarei está aqui: https://upload.wikimedia.org/wikipedia/commons/6/69/NASA-HS201427a-HubbleUltraDeepField2014-20140603.jpg), salve em algum lugar do seu projeto e após isso deixe o Finder aberto, juntamente com o Xcode. No Xcode, você deverá selecionar a pasta **Assets.xcassets** (.xcassets fica oculto), que é onde colocamos as imagens que serão usadas no projeto, e arrastar a imagem do Finder para esta área em branco que surge após selecionar Assets. Agora, nossa imagem está pronta para ser usada no projeto.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

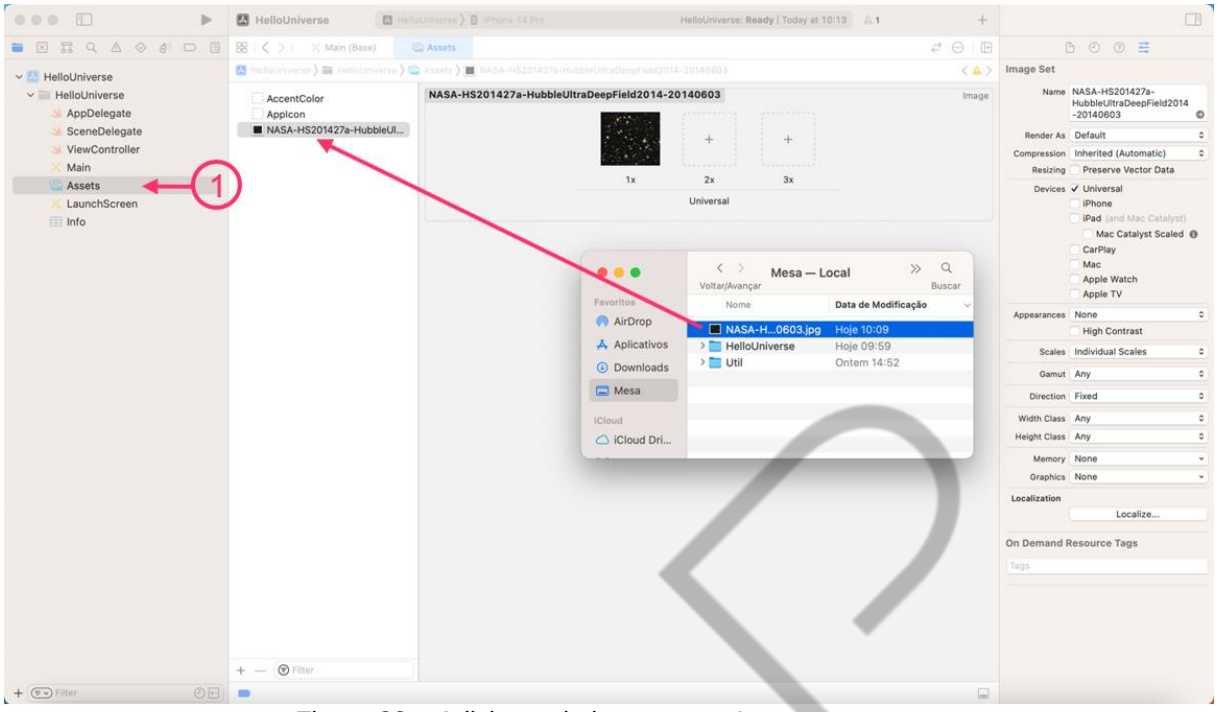


Figura 32 – Adicionando imagem no Assets.xcassets
Fonte: Elaborado pelo autor (2023)

Para que possamos visualizar esta imagem no nosso aplicativo, precisamos adicionar um componente para mostrar imagens. Em iOS, este componente é o **Image View**. Ainda em **Main**, vamos escolher *Image View na Library* (filtrando pelo nome) e adicionar este componente à nossa cena. Manipule o tamanho do *frame* do *Image View (alças de controles quadradas nos cantos do elemento),* de modo que ocupe toda a tela do aplicativo. Nesse momento, você deve ter percebido que este novo componente está por cima da **Label**, ou seja, quando formos adicionar a imagem não conseguiremos ver o **Label** na tela. Para resolver isto, na **Document Outline** selecione o *ImageView* e arraste para cima do **Label**, de modo que fique atrás na tela (elementos mais acima na árvore, se posicionam mais ao fundo na nossa tela).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

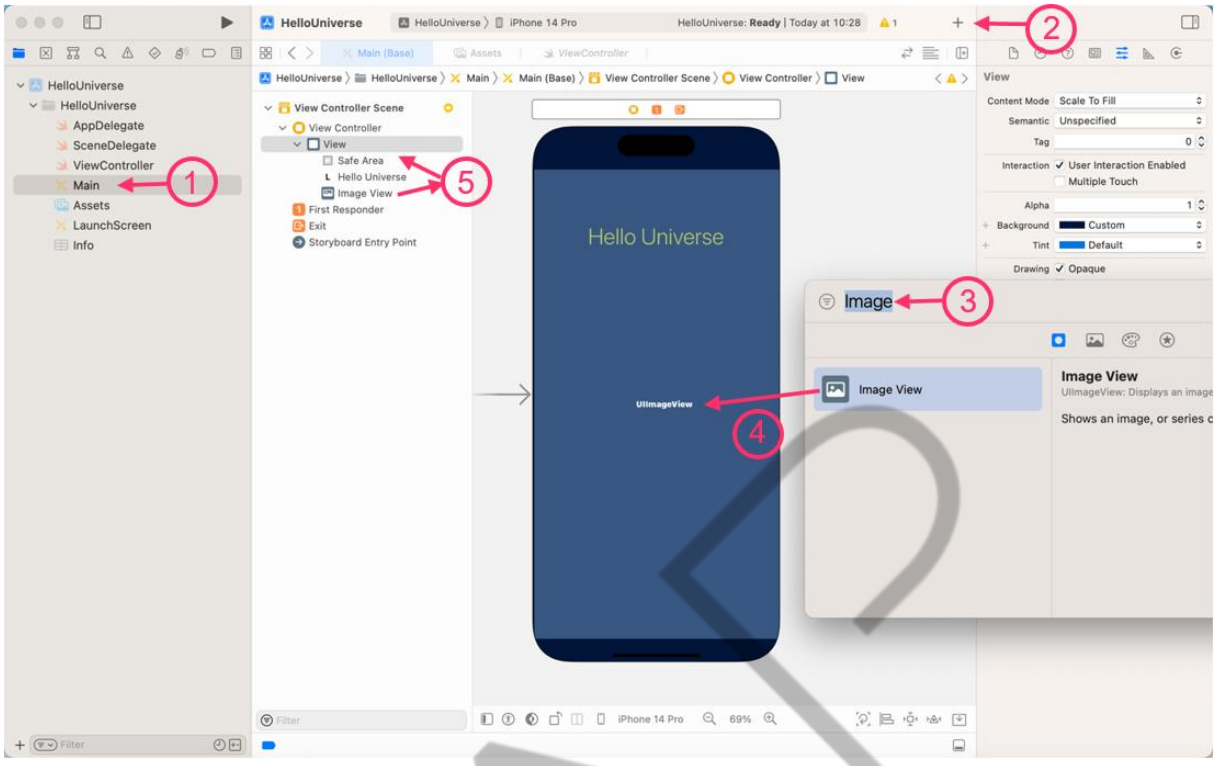


Figura 33 – Adicionando imagem na cena
Fonte: Elaborado pelo autor (2023)

Para finalizar, precisamos atribuir a nossa imagem ao *Image View*. Com ele selecionado, no *Attributes Inspector* clique na opção *Image* e selecione a imagem (no exemplo, o nome original foi **NASA-HS201...jpg**). Fazendo isso, a imagem ocupará parte da tela pois este é o comportamento padrão de uma *Image View*. Para mudarmos este comportamento, vamos trocar para **Aspect Fill**, na opção **Content Mode**. Após isso, nossa imagem ocupará toda a tela. Como esta imagem destaca muitas estrelas, podemos alterar a sua para que o **Label** se destaque mais. Isso é feito alterando a propriedade Alpha, podemos indicar o valor de 0,5.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

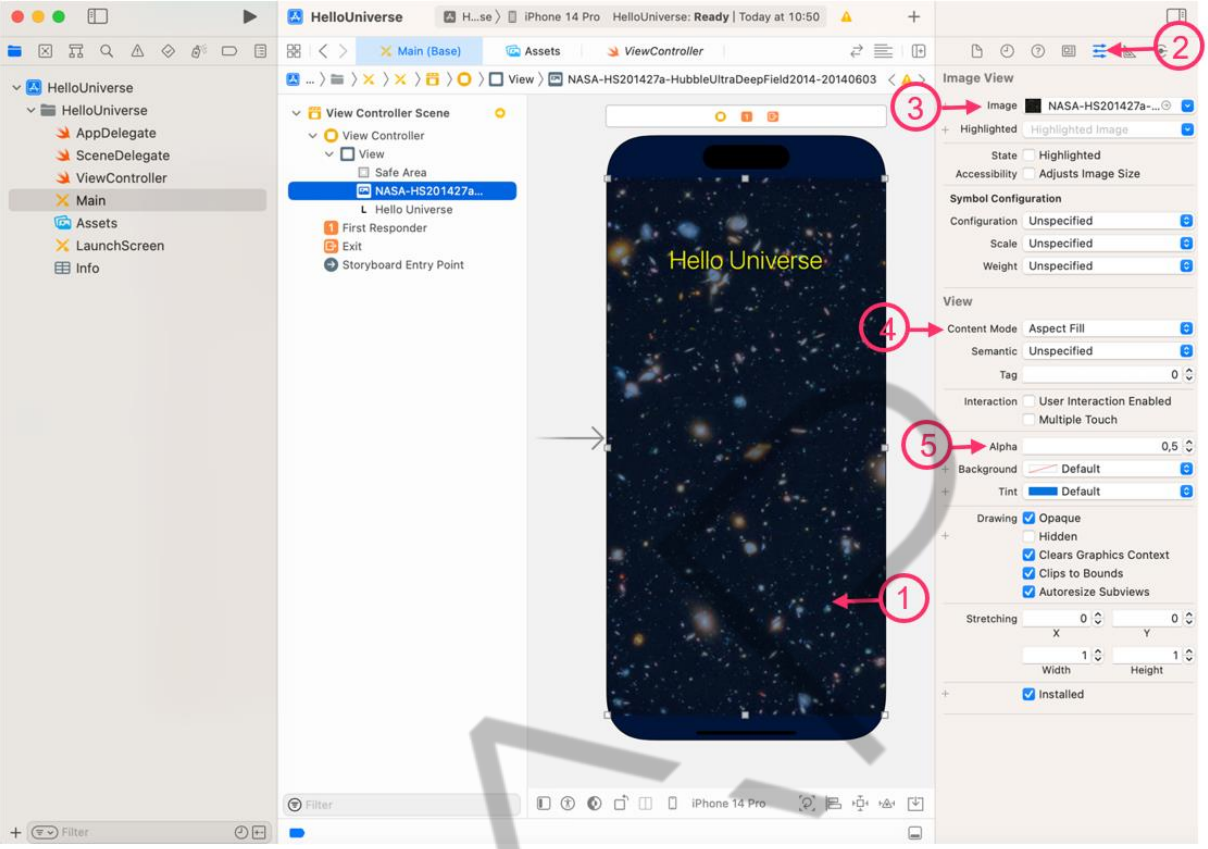


Figura 34 – Configurando Image View
Fonte: Elaborado pelo autor (2023)

Agora, rode seu aplicativo (escolha **iPhone 14 Pro** como simulador) e contemple seu primeiro *Hello World*, ou melhor *Hello Universe* em iOS!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

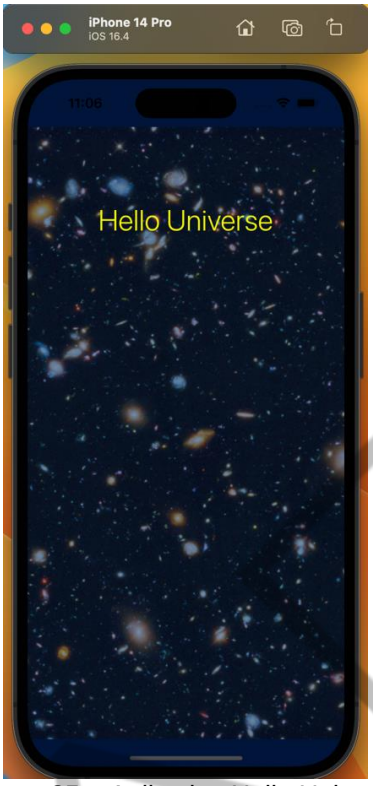


Figura 35 – Aplicativo Hello Universe
Fonte: Elaborado pelo autor (2023)

Para ficar um craque na utilização da interface do Xcode, basta treinar um pouco mais e logo você se familiarizará com todas essas janelas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

APPLE. **Apple Developer Documentation**. 2023. Disponível em: <https://developer.apple.com/documentation>. Acesso em: 21 jun. 2023.

APPLE. **Apple Framework UIKit**. 2023. Disponível em: <https://developer.apple.com/documentation/uikit>. Acesso em: 21 jun. 2023.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com